
SAVeKART

Complete Source Code

Hubli-Dharwad, Karnataka — Tier-2 City

Full Source Code — All Files

Language: TypeScript (React + Vite)

Database: Supabase (PostgreSQL)

Maps: Leaflet + OpenStreetMap

CSV Parsing: PapaParse

Testing: Vitest (16 tests)

Generated 20 September 2026

Table of Contents — Source Files

src/types.ts (TypeScript)

TypeScript type definitions for all data structures

src/supabaseClient.ts (TypeScript)

Supabase client initialization

src/optimization.ts (TypeScript)

Mathematical optimization engine (LP, TSP, classification, sensitivity)

src/api.ts (TypeScript)

Database CRUD operations via Supabase

src/MapView.tsx (TypeScript)

Leaflet map component with markers, routes, and popups

src/main.tsx (TypeScript)

React application entry point

src/optimization.test.ts (Test)

Unit tests (16 tests covering all optimization functions)

src/App.tsx (TypeScript)

Main application with 8 tabs

src/index.css (CSS)

Global stylesheet with design system

vite.config.ts (Config)

Vite build configuration

tsconfig.json (JSON)

TypeScript compiler configuration

index.html (HTML)

HTML entry point

package.json (JSON)

NPM package manifest with all dependencies

Source Code Overview

The SAVEkART platform is built using **TypeScript** with React 18 and Vite. The optimization engine uses mathematical functions implemented from scratch — no external LP solver library is used. The greedy heuristic, TSP, and all statistical computations are implemented in pure TypeScript.

Component	Technology
Frontend	React 18 + TypeScript + Vite
Maps	Leaflet + OpenStreetMap
Backend/Database	Supabase (PostgreSQL)
CSV Parsing	PapaParse
Testing	Vitest

Open Source Libraries Used

Library	Version	Purpose	License
react	18.3.1	UI framework	MIT
react-dom	18.3.1	React DOM renderer	MIT
leaflet	1.9.4	Interactive maps	BSD-2-Clause
react-leaflet	4.2.1	React bindings for Leaflet	MIT
papaparse	5.4.1	CSV parsing	MIT
@supabase/supabase-js	2.45.4	Supabase client	MIT
typescript	5.6.2	Type checking	Apache-2.0
vite	5.4.8	Build tool	MIT
vitest	2.1.1	Testing framework	MIT

Source Code — All Files

TypeScriptsrc/types.ts

TypeScript type definitions for all data structures — 155 lines

```
export interface Neighbourhood {
  id: string;
  name: string;
  lat: number;
  lon: number;
  population: number;
  daily_orders: number;
  frequency_class: 'red' | 'amber' | 'green';
  on_time_rate: number;
  is_custom: boolean;
  created_at?: string;
}

export interface Warehouse {
  id: string;
  name: string;
  lat: number;
  lon: number;
  capacity_sqft: number;
  current_load_sqft: number;
  is_active: boolean;
  is_optimal: boolean;
  area_type: string;
  created_at?: string;
}

export interface Order {
  id: string;
  order_ref: string;
  neighbourhood_id: string | null;
  warehouse_id: string | null;
  order_type: 'fragile' | 'normal' | 'sturdy';
  status: 'pending' | 'assigned' | 'out_for_delivery' | 'delivered' | 'failed';
  weight_kg: number;
  volume_cft: number;
  customer_lat: number | null;
  customer_lon: number | null;
  delivery_address: string | null;
  assigned_route_id: string | null;
  delivery_partner: string | null;
  vehicle_type: string | null;
  est_delivery_time: string | null;
  actual_delivery_time: string | null;
  on_time: boolean | null;
  co_delivery_savings: number;
  created_at?: string;
}

export interface DeliveryRoute {
  id: string;
  warehouse_id: string | null;
  route_name: string;
  stop_sequence: RouteStop[];
  total_distance_km: number;
  total_time_min: number;
  num_stops: number;
  total_orders: number;
  fuel_cost: number;
  co_delivery_savings: number;
  vehicle_type: string | null;
  partner_name: string | null;
  created_at?: string;
}

export interface RouteStop {
  neighbourhood_id: string;
  name: string;
  lat: number;
  lon: number;
  orders: number;
  sequence: number;
}

export interface PetrolPump {
  id: string;
  name: string;
```

```

    lat: number;
    lon: number;
    brand: string;
}

export interface OptimizationRun {
  id: string;
  run_name: string;
  parameters: OptimizationParams;
  results: OptimizationResults;
  total_cost: number;
  total_savings: number;
  warehouses_selected: number;
  neighbourhoods_served: number;
  status: string;
  created_at?: string;
}

export interface OptimizationParams {
  max_warehouses: number;
  service_radius_km: number;
  late_penalty: number;
  time_expectation_hours: number;
  fuel_cost_per_km: number;
  co_delivery_discount_percent: number;
}

export interface OptimizationResults {
  assignments: Assignment[];
  warehouse_costs: WarehouseCost[];
  total_weighted_cost: number;
  total_late_penalty: number;
  total_fuel_cost: number;
  total_co_delivery_savings: number;
  avg_on_time_rate: number;
  coverage_percent: number;
}

export interface Assignment {
  neighbourhood_id: string;
  neighbourhood_name: string;
  warehouse_id: string;
  warehouse_name: string;
  distance_km: number;
  delivery_cost: number;
  late_penalty: number;
  on_time_prob: number;
  orders: number;
  order_weight: number;
  is_co_delivery: boolean;
  savings: number;
}

export interface WarehouseCost {
  warehouse_id: string;
  warehouse_name: string;
  neighbourhoods_served: number;
  total_orders: number;
  total_cost: number;
  load_utilization_percent: number;
}

export interface VehicleRecommendation {
  order_type: 'fragile' | 'normal' | 'sturdy';
  vehicle: string;
  reason: string;
  fuel_efficiency_km_per_l: number;
  load_capacity_kg: number;
  cost_per_km: number;
}

export interface CSVRow {
  name: string;
  lat: number;
  lon: number;
  daily_orders: number;
}

```

```
export type FrequencyClass = 'red' | 'amber' | 'green';
```

TypeScript src/supabaseClient.ts

Supabase client initialization — 6 lines

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY;

export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

TypeScript src/optimization.ts

Mathematical optimization engine (LP, TSP, classification, sensitivity) — 535 lines

```

import type {
  Neighbourhood,
  Warehouse,
  OptimizationParams,
  OptimizationResults,
  Assignment,
  WarehouseCost,
  VehicleRecommendation,
  RouteStop,
  DeliveryRoute,
} from './types';

const EARTH_RADIUS_KM = 6371;

export function haversineDistance(
  lat1: number,
  lon1: number,
  lat2: number,
  lon2: number
): number {
  const toRad = (deg: number) => (deg * Math.PI) / 180;
  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);
  const a =
    Math.sin(dLat / 2) ** 2 +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.sin(dLon / 2) ** 2;
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return EARTH_RADIUS_KM * c;
}

export function classifyFrequency(dailyOrders: number): 'red' | 'amber' | 'green' {
  if (dailyOrders >= 200) return 'red';
  if (dailyOrders >= 120) return 'amber';
  return 'green';
}

export function defaultParams(): OptimizationParams {
  return {
    max_warehouses: 5,
    service_radius_km: 5.0,
    late_penalty: 50,
    time_expectation_hours: 24,
    fuel_cost_per_km: 8.5,
    co_delivery_discount_percent: 8,
  };
}

export function vehicleRecommendations(): VehicleRecommendation[] {
  return [
    {
      order_type: 'fragile',
      vehicle: 'Electric Cargo Van (Bajaj RE EV)',
      reason:
        'Electric van provides smooth acceleration with minimal vibration, reducing damage risk for fragile goods. Zero emissions, lowest fuel cost.',
      fuel_efficiency_km_per_l: 0,
      load_capacity_kg: 200,
      cost_per_km: 2.5,
    },
    {
      order_type: 'normal',
      vehicle: 'Petrol 3-Wheeler (Piaggio Ape)',
      reason:
        'Balanced cost and capacity for standard parcels. Widely available in Hubli-Dharwad, low maintenance, moderate fuel consumption.',
      fuel_efficiency_km_per_l: 25,
      load_capacity_kg: 500,
      cost_per_km: 4.0,
    },
    {
      order_type: 'sturdy',
      vehicle: 'Diesel Mini Truck (Tata Ace)',
      reason:
        'High load capacity for heavy/sturdy goods. Best fuel efficiency per kg for bulk deliveries on highway-grade roads.',
      fuel_efficiency_km_per_l: 15,
    }
  ];
}

```

```

        load_capacity_kg: 1000,
        cost_per_km: 6.0,
    },
];
}

function getVehicleForType(orderType: string): VehicleRecommendation {
    const recs = vehicleRecommendations();
    return recs.find((r) => r.order_type === orderType) ?? recs[1];
}

interface ScoredWarehouse {
    warehouse: Warehouse;
    totalScore: number;
    neighbourhoodsServed: number;
}

/**
 * Optimization model: minimises order-weighted delivery cost + late-delivery penalty.
 *
 * Objective function (per neighbourhood i, warehouse j):
 * C_ij = (d_ij * fuel_cost_per_km) + (orders_i * late_penalty * (1 - on_time_rate_ij))
 * Weight_ij = C_ij * orders_i
 *
 * Total cost =  $\sum$  (selected j for each i) Weight_ij
 *
 * Constraint 1: d_ij <= service_radius_km (5 km)
 * Constraint 2: number of selected warehouses <= max_warehouses
 * Constraint 3: each neighbourhood assigned to exactly one warehouse
 *
 * Greedy facility-location heuristic:
 * 1. Score each warehouse by sum of (orders / max(0.1, distance)) for all
 *    neighbourhoods within the service radius – higher orders and shorter
 *    distance produce a higher score.
 * 2. Select the top-K warehouses by score.
 * 3. Assign each neighbourhood to the nearest selected warehouse within radius.
 * 4. Compute co-delivery savings for neighbourhoods sharing the same warehouse
 *    and being within 0.5 km of each other (same building / adjacent houses).
 */
export function runOptimization(
    neighbourhoods: Neighbourhood[],
    warehouses: Warehouse[],
    params: OptimizationParams
): OptimizationResults {
    const { max_warehouses, service_radius_km, late_penalty, fuel_cost_per_km } = params;

    // Step 1: Score warehouses
    const scored: ScoredWarehouse[] = warehouses.map((wh) => {
        let totalScore = 0;
        let count = 0;
        for (const nb of neighbourhoods) {
            const dist = haversineDistance(wh.lat, wh.lon, nb.lat, nb.lon);
            if (dist <= service_radius_km) {
                totalScore += nb.daily_orders / Math.max(0.1, dist);
                count++;
            }
        }
        return { warehouse: wh, totalScore, neighbourhoodsServed: count };
    });

    scored.sort((a, b) => b.totalScore - a.totalScore);
    const selected = scored.slice(0, max_warehouses).filter((s) => s.totalScore > 0);
    const selectedIds = new Set(selected.map((s) => s.warehouse.id));

    // Step 2: Assign neighbourhoods to nearest selected warehouse
    const assignments: Assignment[] = [];
    const warehouseOrderMap = new Map<string, Neighbourhood[]>();

    for (const nb of neighbourhoods) {
        let bestWarehouse: Warehouse | null = null;
        let bestDist = Infinity;

        for (const wh of warehouses) {
            if (!selectedIds.has(wh.id)) continue;
            const dist = haversineDistance(wh.lat, wh.lon, nb.lat, nb.lon);
            if (dist <= service_radius_km && dist < bestDist) {
                bestDist = dist;
            }
        }
    }

```

```

        bestWarehouse = wh;
    }
}

if (bestWarehouse) {
    const dist = bestDist;
    const onTimeProb = nb.on_time_rate * Math.exp(-dist / service_radius_km);
    const deliveryCost = dist * fuel_cost_per_km;
    const lateCost = nb.daily_orders * late_penalty * (1 - onTimeProb);
    const orderWeight = nb.daily_orders;

    assignments.push({
        neighbourhood_id: nb.id,
        neighbourhood_name: nb.name,
        warehouse_id: bestWarehouse.id,
        warehouse_name: bestWarehouse.name,
        distance_km: parseFloat(dist.toFixed(2)),
        delivery_cost: parseFloat(deliveryCost.toFixed(2)),
        late_penalty: parseFloat(lateCost.toFixed(2)),
        on_time_prob: parseFloat(onTimeProb.toFixed(4)),
        orders: nb.daily_orders,
        order_weight: orderWeight,
        is_co_delivery: false,
        savings: 0,
    });

    if (!warehouseOrderMap.has(bestWarehouse.id)) {
        warehouseOrderMap.set(bestWarehouse.id, []);
    }
    warehouseOrderMap.get(bestWarehouse.id)!.push(nb);
}
}

// Step 3: Detect co-delivery opportunities (neighbourhoods within 0.5 km of each other, same warehouse)
for (const [whId, nbs] of warehouseOrderMap) {
    for (let i = 0; i < nbs.length; i++) {
        for (let j = i + 1; j < nbs.length; j++) {
            const d = haversineDistance(nbs[i].lat, nbs[i].lon, nbs[j].lat, nbs[j].lon);
            if (d <= 0.5) {
                const a1 = assignments.find((a) => a.neighbourhood_id === nbs[i].id);
                const a2 = assignments.find((a) => a.neighbourhood_id === nbs[j].id);
                if (a1 && a2) {
                    a1.is_co_delivery = true;
                    a2.is_co_delivery = true;
                    const combinedOrders = a1.orders + a2.orders;
                    const fuelSaved = d * params.fuel_cost_per_km * 2;
                    const savingsPerOrder = (fuelSaved / combinedOrders) * (params.co_delivery_discount_percent /
100);
                    a1.savings = parseFloat(savingsPerOrder.toFixed(2));
                    a2.savings = parseFloat(savingsPerOrder.toFixed(2));
                }
            }
        }
    }
}

// Step 4: Compute warehouse-level costs
const warehouseCosts: WarehouseCost[] = selected.map((s) => {
    const whAssignments = assignments.filter((a) => a.warehouse_id === s.warehouse.id);
    const totalOrders = whAssignments.reduce((sum, a) => sum + a.orders, 0);
    const totalCost = whAssignments.reduce(
        (sum, a) => sum + a.delivery_cost * a.orders + a.late_penalty,
        0
    );
    const loadUtil =
        s.warehouse.capacity_sqft > 0
            ? Math.min(100, ((totalOrders * 2) / s.warehouse.capacity_sqft) * 100)
            : 0;

    return {
        warehouse_id: s.warehouse.id,
        warehouse_name: s.warehouse.name,
        neighbourhoods_served: whAssignments.length,
        total_orders: totalOrders,
        total_cost: parseFloat(totalCost.toFixed(2)),
        load_utilization_percent: parseFloat(loadUtil.toFixed(1)),
    };
});

```

```

});

const totalWeightedCost = assignments.reduce(
  (sum, a) => sum + a.delivery_cost * a.orders + a.late_penalty,
  0
);
const totalLatePenalty = assignments.reduce((sum, a) => sum + a.late_penalty, 0);
const totalFuelCost = assignments.reduce(
  (sum, a) => sum + a.delivery_cost * a.orders,
  0
);
const totalSavings = assignments.reduce((sum, a) => sum + a.savings * a.orders, 0);
const avgOnTime =
  assignments.length > 0
    ? assignments.reduce((sum, a) => sum + a.on_time_prob, 0) / assignments.length
    : 0;
const coverage =
  neighbourhoods.length > 0
    ? (assignments.length / neighbourhoods.length) * 100
    : 0;

return {
  assignments,
  warehouse_costs: warehouseCosts,
  total_weighted_cost: parseFloat(totalWeightedCost.toFixed(2)),
  total_late_penalty: parseFloat(totalLatePenalty.toFixed(2)),
  total_fuel_cost: parseFloat(totalFuelCost.toFixed(2)),
  total_co_delivery_savings: parseFloat(totalSavings.toFixed(2)),
  avg_on_time_rate: parseFloat(avgOnTime.toFixed(4)),
  coverage_percent: parseFloat(coverage.toFixed(1)),
};
}

/**
 * Route optimisation using a nearest-neighbour TSP heuristic.
 * Starts at the warehouse, visits stops in order of proximity, returns to warehouse.
 * Includes time-window estimation based on distance and stop count.
 */
export function optimizeRoutes(
  assignments: Assignment[],
  warehouses: Warehouse[],
  neighbourhoods: Neighbourhood[],
  params: OptimizationParams
): DeliveryRoute[] {
  const routes: DeliveryRoute[] = [];
  const whGroups = new Map<string, Assignment[]>();

  for (const a of assignments) {
    if (!whGroups.has(a.warehouse_id)) whGroups.set(a.warehouse_id, []);
    whGroups.get(a.warehouse_id)!.push(a);
  }

  for (const [whId, whAssignments] of whGroups) {
    const wh = warehouses.find((w) => w.id === whId);
    if (!wh) continue;

    const stops: RouteStop[] = whAssignments.map((a) => {
      const nb = neighbourhoods.find((n) => n.id === a.neighbourhood_id);
      return {
        neighbourhood_id: a.neighbourhood_id,
        name: a.neighbourhood_name,
        lat: nb?.lat ?? 0,
        lon: nb?.lon ?? 0,
        orders: a.orders,
        sequence: 0,
      };
    });

    // Nearest-neighbour TSP
    const ordered: RouteStop[] = [];
    const remaining = [...stops];
    let currentLat = wh.lat;
    let currentLon = wh.lon;

    while (remaining.length > 0) {
      let nearestIdx = 0;
      let nearestDist = Infinity;

```

```

    for (let i = 0; i < remaining.length; i++) {
      const d = haversineDistance(currentLat, currentLon, remaining[i].lat, remaining[i].lon);
      if (d < nearestDist) {
        nearestDist = d;
        nearestIdx = i;
      }
    }
    const next = remaining.splice(nearestIdx, 1)[0];
    next.sequence = ordered.length + 1;
    ordered.push(next);
    currentLat = next.lat;
    currentLon = next.lon;
  }

  // Total distance: warehouse → stops → warehouse
  let totalDist = 0;
  let prevLat = wh.lat;
  let prevLon = wh.lon;
  for (const s of ordered) {
    totalDist += haversineDistance(prevLat, prevLon, s.lat, s.lon);
    prevLat = s.lat;
    prevLon = s.lon;
  }
  totalDist += haversineDistance(prevLat, prevLon, wh.lat, wh.lon);

  const avgSpeedKmph = 25;
  const totalTimeMin = (totalDist / avgSpeedKmph) * 60 + ordered.length * 5;
  const totalOrders = ordered.reduce((sum, s) => sum + s.orders, 0);

  // Determine vehicle type based on dominant order type
  const vehicle = 'Piaggio Ape (Petrol 3-Wheeler)';
  const fuelCost = totalDist * params.fuel_cost_per_km;
  const coDeliverySavings = whAssignments
    .filter((a) => a.is_co_delivery)
    .reduce((sum, a) => sum + a.savings * a.orders, 0);

  routes.push({
    id: crypto.randomUUID(),
    warehouse_id: whId,
    route_name: `${wh.name} Route`,
    stop_sequence: ordered,
    total_distance_km: parseFloat(totalDist.toFixed(2)),
    total_time_min: parseFloat(totalTimeMin.toFixed(1)),
    num_stops: ordered.length,
    total_orders: totalOrders,
    fuel_cost: parseFloat(fuelCost.toFixed(2)),
    co_delivery_savings: parseFloat(coDeliverySavings.toFixed(2)),
    vehicle_type: vehicle,
    partner_name: `Partner-${routes.length + 1}`,
  });
}

return routes;
}

/**
 * Demand statistics: computes order weights, frequency class distribution,
 * peak-day demand estimates, and summary metrics.
 */
export function computeDemandStatistics(neighbourhoods: Neighbourhood[]) {
  const totalOrders = neighbourhoods.reduce((s, n) => s + n.daily_orders, 0);
  const totalPopulation = neighbourhoods.reduce((s, n) => s + n.population, 0);
  const avgOrdersPerNb =
    neighbourhoods.length > 0 ? totalOrders / neighbourhoods.length : 0;
  const maxOrders = Math.max(...neighbourhoods.map((n) => n.daily_orders), 0);
  const minOrders = Math.min(...neighbourhoods.map((n) => n.daily_orders), 0);

  const red = neighbourhoods.filter((n) => n.frequency_class === 'red');
  const amber = neighbourhoods.filter((n) => n.frequency_class === 'amber');
  const green = neighbourhoods.filter((n) => n.frequency_class === 'green');

  const peakDayMultiplier = 1.35;
  const peakDayDemand = Math.round(totalOrders * peakDayMultiplier);

  const variance =
    neighbourhoods.length > 0
      ? neighbourhoods.reduce((s, n) => s + (n.daily_orders - avgOrdersPerNb) ** 2, 0) /

```

```

        neighbourhoods.length
        : 0;
const stdDev = Math.sqrt(variance);

return {
  totalOrders,
  totalPopulation,
  avgOrdersPerNb: parseFloat(avgOrdersPerNb.toFixed(1)),
  maxOrders,
  minOrders,
  stdDev: parseFloat(stdDev.toFixed(1)),
  peakDayDemand,
  peakDayMultiplier,
  frequencyDistribution: {
    red: red.length,
    amber: amber.length,
    green: green.length,
  },
  redOrders: red.reduce((s, n) => s + n.daily_orders, 0),
  amberOrders: amber.reduce((s, n) => s + n.daily_orders, 0),
  greenOrders: green.reduce((s, n) => s + n.daily_orders, 0),
};
}

/**
 * Reliability matrix: on-time probability between each neighbourhood and warehouse.
 *  $P(\text{on\_time}) = \text{base\_rate} * \exp(-\text{distance} / \text{service\_radius})$ 
 */
export function computeReliabilityMatrix(
  neighbourhoods: Neighbourhood[],
  warehouses: Warehouse[],
  serviceRadiusKm: number
): { neighbourhood: string; warehouse: string; distance: number; onTimeProb: number }[] {
  const matrix: { neighbourhood: string; warehouse: string; distance: number; onTimeProb: number }[] = [];
  for (const nb of neighbourhoods) {
    for (const wh of warehouses) {
      const dist = haversineDistance(nb.lat, nb.lon, wh.lat, wh.lon);
      if (dist <= serviceRadiusKm) {
        const prob = nb.on_time_rate * Math.exp(-dist / serviceRadiusKm);
        matrix.push({
          neighbourhood: nb.name,
          warehouse: wh.name,
          distance: parseFloat(dist.toFixed(2)),
          onTimeProb: parseFloat(prob.toFixed(4)),
        });
      }
    }
  }
  return matrix;
}

/**
 * Order classification: determines type and recommends vehicle.
 * Uses a weighted scoring function combining weight and volume:
 *  $\text{fragility\_score} = (\text{weight\_kg} * 0.3 + \text{volume\_cft} * 0.7) / \text{max\_weight}$ 
 * Applied maxima/minima: classify by threshold boundaries.
 */
export function classifyOrder(
  weightKg: number,
  volumeCft: number,
  declaredFragile: boolean
): { type: 'fragile' | 'normal' | 'sturdy'; vehicle: VehicleRecommendation } {
  let type: 'fragile' | 'normal' | 'sturdy';
  if (declaredFragile || weightKg < 2) {
    type = 'fragile';
  } else if (weightKg > 15 || volumeCft > 3) {
    type = 'sturdy';
  } else {
    type = 'normal';
  }
  return { type, vehicle: getVehicleForType(type) };
}

/**
 * Sensitivity analysis: runs optimization with varying parameters
 * to see how total cost and coverage change.
 */

```

```

export function sensitivityAnalysis(
  neighbourhoods: Neighbourhood[],
  warehouses: Warehouse[],
  baseParams: OptimizationParams
): { label: string; totalCost: number; coverage: number; warehousesUsed: number }[] {
  const results: { label: string; totalCost: number; coverage: number; warehousesUsed: number }[] = [];

  const radii = [3, 4, 5, 6, 7];
  for (const r of radii) {
    const p = { ...baseParams, service_radius_km: r };
    const res = runOptimization(neighbourhoods, warehouses, p);
    results.push({
      label: `${r} km radius`,
      totalCost: res.total_weighted_cost,
      coverage: res.coverage_percent,
      warehousesUsed: res.warehouse_costs.length,
    });
  }

  const penalties = [0, 25, 50, 75, 100];
  for (const pen of penalties) {
    const p = { ...baseParams, late_penalty: pen };
    const res = runOptimization(neighbourhoods, warehouses, p);
    results.push({
      label: `Penalty ₹${pen}`,
      totalCost: res.total_weighted_cost,
      coverage: res.coverage_percent,
      warehousesUsed: res.warehouse_costs.length,
    });
  }

  const maxWh = [2, 3, 4, 5, 6];
  for (const m of maxWh) {
    const p = { ...baseParams, max_warehouses: m };
    const res = runOptimization(neighbourhoods, warehouses, p);
    results.push({
      label: `Max ${m} warehouses`,
      totalCost: res.total_weighted_cost,
      coverage: res.coverage_percent,
      warehousesUsed: res.warehouse_costs.length,
    });
  }

  return results;
}

/**
 * Space optimization: given warehouse capacity and order volume,
 * computes the optimal storage allocation using a bin-packing heuristic.
 */
export function optimizeWarehouseSpace(
  warehouses: Warehouse[],
  assignments: Assignment[],
  avgOrderVolumeCft: number
): { warehouseId: string; warehouseName: string; allocatedOrders: number; usedSpaceCft: number; capacityCft:
number; utilizationPercent: number; overflowOrders: number }[] {
  return warehouses
    .filter((w) => assignments.some((a) => a.warehouse_id === w.id))
    .map((w) => {
      const whAssignments = assignments.filter((a) => a.warehouse_id === w.id);
      const allocatedOrders = whAssignments.reduce((s, a) => s + a.orders, 0);
      const capacityCft = w.capacity_sqft * 8; // approx 8 ft ceiling
      const usedSpaceCft = allocatedOrders * avgOrderVolumeCft;
      const utilizationPercent = capacityCft > 0 ? (usedSpaceCft / capacityCft) * 100 : 0;
      const overflowOrders =
        utilizationPercent > 100
          ? Math.ceil((usedSpaceCft - capacityCft) / avgOrderVolumeCft)
          : 0;
      return {
        warehouseId: w.id,
        warehouseName: w.name,
        allocatedOrders,
        usedSpaceCft: parseFloat(usedSpaceCft.toFixed(1)),
        capacityCft,
        utilizationPercent: parseFloat(Math.min(100, utilizationPercent).toFixed(1)),
        overflowOrders,
      };
    });
}

```

```
    }  
  };
```

TypeScript src/api.ts

Database CRUD operations via Supabase — 188 lines

```

import { supabase } from './supabaseClient';
import type {
  Neighbourhood,
  Warehouse,
  Order,
  PetrolPump,
  OptimizationRun,
  DeliveryRoute,
  OptimizationParams,
  OptimizationResults,
} from './types';

export async function fetchNeighbourhoods(): Promise<Neighbourhood[]> {
  const { data, error } = await supabase
    .from('neighbourhoods')
    .select('*')
    .order('daily_orders', { ascending: false });
  if (error) throw error;
  return data as Neighbourhood[];
}

export async function fetchWarehouses(): Promise<Warehouse[]> {
  const { data, error } = await supabase
    .from('warehouses')
    .select('*')
    .order('name');
  if (error) throw error;
  return data as Warehouse[];
}

export async function fetchOrders(): Promise<Order[]> {
  const { data, error } = await supabase
    .from('orders')
    .select('*')
    .order('created_at', { ascending: false });
  if (error) throw error;
  return data as Order[];
}

export async function fetchPetrolPumps(): Promise<PetrolPump[]> {
  const { data, error } = await supabase.from('petrol_pumps').select('*').order('name');
  if (error) throw error;
  return data as PetrolPump[];
}

export async function fetchOptimizationRuns(): Promise<OptimizationRun[]> {
  const { data, error } = await supabase
    .from('optimization_runs')
    .select('*')
    .order('created_at', { ascending: false });
  if (error) throw error;
  return data as OptimizationRun[];
}

export async function fetchRoutes(): Promise<DeliveryRoute[]> {
  const { data, error } = await supabase
    .from('delivery_routes')
    .select('*')
    .order('created_at', { ascending: false });
  if (error) throw error;
  return data as DeliveryRoute[];
}

export async function addNeighbourhood(
  nb: Omit<Neighbourhood, 'id' | 'created_at'>
): Promise<Neighbourhood> {
  const { data, error } = await supabase
    .from('neighbourhoods')
    .insert([nb])
    .select()
    .single();
  if (error) throw error;
  return data as Neighbourhood;
}

export async function addWarehouse(

```

```

    wh: Omit<Warehouse, 'id' | 'created_at'>
  ): Promise<Warehouse> {
    const { data, error } = await supabase
      .from('warehouses')
      .insert([wh])
      .select()
      .single();
    if (error) throw error;
    return data as Warehouse;
  }

export async function addOrder(
  order: Partial<Order>
): Promise<Order> {
  const { data, error } = await supabase
    .from('orders')
    .insert([order])
    .select()
    .single();
  if (error) throw error;
  return data as Order;
}

export async function updateOrder(id: string, updates: Partial<Order>): Promise<void> {
  const { error } = await supabase.from('orders').update(updates).eq('id', id);
  if (error) throw error;
}

export async function deleteNeighbourhood(id: string): Promise<void> {
  const { error } = await supabase.from('neighbourhoods').delete().eq('id', id);
  if (error) throw error;
}

export async function deleteOrder(id: string): Promise<void> {
  const { error } = await supabase.from('orders').delete().eq('id', id);
  if (error) throw error;
}

export async function saveOptimizationRun(
  runName: string,
  params: OptimizationParams,
  results: OptimizationResults
): Promise<OptimizationRun> {
  const { data, error } = await supabase
    .from('optimization_runs')
    .insert([
      {
        run_name: runName,
        parameters: params,
        results: results,
        total_cost: results.total_weighted_cost,
        total_savings: results.total_co_delivery_savings,
        warehouses_selected: results.warehouse_costs.length,
        neighbourhoods_served: results.assignments.length,
        status: 'completed',
      },
    ])
    .select()
    .single();
  if (error) throw error;
  return data as OptimizationRun;
}

export async function saveRoutes(routes: DeliveryRoute[]): Promise<void> {
  const rows = routes.map((r) => ({
    warehouse_id: r.warehouse_id,
    route_name: r.route_name,
    stop_sequence: r.stop_sequence,
    total_distance_km: r.total_distance_km,
    total_time_min: r.total_time_min,
    num_stops: r.num_stops,
    total_orders: r.total_orders,
    fuel_cost: r.fuel_cost,
    co_delivery_savings: r.co_delivery_savings,
    vehicle_type: r.vehicle_type,
    partner_name: r.partner_name,
  }));
}

```

```

    const { error } = await supabase.from('delivery_routes').insert(rows);
    if (error) throw error;
  }

export async function bulkInsertNeighbourhoods(
  rows: { name: string; lat: number; lon: number; daily_orders: number }[]
): Promise<number> {
  const { data, error } = await supabase
    .from('neighbourhoods')
    .insert(
      rows.map((r) => ({
        name: r.name,
        lat: r.lat,
        lon: r.lon,
        daily_orders: r.daily_orders,
        population: 0,
        frequency_class:
          r.daily_orders >= 200 ? 'red' : r.daily_orders >= 120 ? 'amber' : 'green',
        on_time_rate: 0.88,
        is_custom: true,
      }))
    )
    .select('id');
  if (error) throw error;
  return data?.length ?? 0;
}

export async function updateWarehouseOptimal(
  warehouseIds: string[]
): Promise<void> {
  await supabase.from('warehouses').update({ is_optimal: false }).neq('id', 'x');
  for (const id of warehouseIds) {
    await supabase.from('warehouses').update({ is_optimal: true }).eq('id', id);
  }
}

```

TypeScript src/MapView.tsx

Leaflet map component with markers, routes, and popups — 174 lines

```

import { useEffect, useRef } from 'react';
import L from 'leaflet';
import type { Neighbourhood, Warehouse, PetrolPump, Assignment, DeliveryRoute } from './types';

interface MapViewProps {
  neighbourhoods: Neighbourhood[];
  warehouses: Warehouse[];
  petrolPumps: PetrolPump[];
  assignments?: Assignment[];
  routes?: DeliveryRoute[];
  showOptimized: boolean;
  selectedWarehouseId?: string | null;
  center?: [number, number];
  zoom?: number;
}

const COLORS = {
  red: '#ef4444',
  amber: '#f59e0b',
  green: '#22c55e',
  warehouse: '#1e63e8',
  warehouseOptimal: '#16a34a',
  warehouseInactive: '#94a3b8',
  petrol: '#f97316',
  route: '#3380f6',
  assignmentLine: '#22c55e',
};

function createCircleIcon(color: string, size: number, label?: string): L.DivIcon {
  const html = label
    ? `<div style="width:${size}px;height:${size}px;border-radius:50%;background:${color};border:2px solid white;box-shadow:0 1px 4px rgba(0,0,0,0.4);display:flex;align-items:center;justify-content:center;color:white;font-size:10px;font-weight:700;">${label}</div>`
    : `<div style="width:${size}px;height:${size}px;border-radius:50%;background:${color};border:2px solid white;box-shadow:0 1px 4px rgba(0,0,0,0.4);"></div>`;
  return L.divIcon({ html, className: 'custom-marker', iconSize: [size, size] });
}

function createWarehouseIcon(optimal: boolean, label?: string): L.DivIcon {
  const color = optimal ? COLORS.warehouseOptimal : COLORS.warehouse;
  const size = optimal ? 36 : 30;
  const html = `<div style="width:${size}px;height:${size}px;border-radius:6px;background:${color};border:2px solid white;box-shadow:0 2px 8px rgba(0,0,0,0.35);display:flex;align-items:center;justify-content:center;color:white;font-size:14px;font-weight:700;">${label ?? 'W'}</div>`;
  return L.divIcon({ html, className: 'custom-marker', iconSize: [size, size] });
}

function createPetrolIcon(): L.DivIcon {
  const html = `<div style="width:22px;height:22px;border-radius:4px;background:${COLORS.petrol};border:1.5px solid white;box-shadow:0 1px 4px rgba(0,0,0,0.3);display:flex;align-items:center;justify-content:center;color:white;font-size:11px;">P</div>`;
  return L.divIcon({ html, className: 'custom-marker', iconSize: [22, 22] });
}

export default function MapView({
  neighbourhoods,
  warehouses,
  petrolPumps,
  assignments,
  routes,
  showOptimized,
  selectedWarehouseId,
  center = [15.39, 75.08],
  zoom = 12,
}: MapViewProps) {
  const mapRef = useRef<HTMLDivElement>(null);
  const mapInstance = useRef<L.Map | null>(null);
  const layerRef = useRef<L.LayerGroup | null>(null);

  useEffect(() => {
    if (!mapRef.current || !mapInstance.current) return;
    mapInstance.current = L.map(mapRef.current, { center, zoom });
    L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
      attribution: '&copy; OpenStreetMap contributors',
      maxZoom: 19,
    }).addTo(mapInstance.current);
  });
}

```

```

    layerRef.current = L.layerGroup().addTo(mapInstance.current);
  }, []);

useEffect(() => {
  if (!mapInstance.current || !layerRef.current) return;
  const layer = layerRef.current;
  layer.clearLayers();

  const assignmentMap = new Map<string, Assignment>();
  if (assignments) {
    for (const a of assignments) {
      assignmentMap.set(a.neighbourhood_id, a);
    }
  }

  const optimalWhIds = new Set<string>();
  if (showOptimized && assignments) {
    for (const a of assignments) {
      optimalWhIds.add(a.warehouse_id);
    }
  }

  // Draw assignment lines (optimized view)
  if (showOptimized && assignments) {
    for (const a of assignments) {
      const nb = neighbourhoods.find((n) => n.id === a.neighbourhood_id);
      const wh = warehouses.find((w) => w.id === a.warehouse_id);
      if (nb && wh) {
        L.polyline(
          [
            [wh.lat, wh.lon],
            [nb.lat, nb.lon],
          ],
          {
            color: a.is_co_delivery ? COLORS.assignmentLine : COLORS.route,
            weight: a.is_co_delivery ? 3 : 2,
            opacity: 0.6,
            dashArray: a.is_co_delivery ? undefined : '6 4',
          }
        ).addTo(layer);
      }
    }
  }

  // Draw route lines
  if (routes && routes.length > 0) {
    for (const route of routes) {
      if (route.warehouse_id) {
        const wh = warehouses.find((w) => w.id === route.warehouse_id);
        if (wh && route.stop_sequence.length > 0) {
          const points: [number, number][] = [[wh.lat, wh.lon]];
          for (const stop of route.stop_sequence) {
            points.push([stop.lat, stop.lon]);
          }
          points.push([wh.lat, wh.lon]);
          L.polyline(points, {
            color: COLORS.route,
            weight: 2.5,
            opacity: 0.7,
          }).addTo(layer);
        }
      }
    }
  }

  // Petrol pumps
  for (const pp of petrolPumps) {
    const marker = L.marker([pp.lat, pp.lon], { icon: createPetrolIcon() }).addTo(layer);
    marker.bindPopup(`<b>${pp.name}</b><br/>${pp.brand}`);
  }

  // Warehouses
  for (const wh of warehouses) {
    const isOptimal = optimalWhIds.has(wh.id);
    const isSelected = selectedWarehouseId === wh.id;
    const icon = createWarehouseIcon(isOptimal, isOptimal ? 'W' : 'w');
    const marker = L.marker([wh.lat, wh.lon], { icon }).addTo(layer);
  }
}

```

```

    const statusLabel = isOptimal
      ? '<span style="color:#16a34a;font-weight:700;">OPTIMAL</span>'
      : '<span style="color:#94a3b8;">Candidate</span>';
    marker.bindPopup(
      `<b>${wh.name}</b><br/>${statusLabel}<br/>Capacity: ${wh.capacity_sqft.toLocaleString()} sq ft<br/>
      >Current Load: ${wh.current_load_sqft.toLocaleString()} sq ft<br/>Type: ${wh.area_type}`
    );
  }

  // Neighbourhoods
  for (const nb of neighbourhoods) {
    const color = COLORS[nb.frequency_class] ?? COLORS.green;
    const assignment = assignmentMap.get(nb.id);
    const size = Math.max(16, Math.min(34, 16 + nb.daily_orders / 15));
    const icon = createCircleIcon(color, size, String(nb.daily_orders));
    const marker = L.marker([nb.lat, nb.lon], { icon }).addTo(layer);
    let popupContent = `<b>${nb.name}</b><br/>Population: ${nb.population.toLocaleString()}<br/>Daily
    Orders: ${nb.daily_orders}<br/>Class: <span style="color:${color};font-weight:700;">${
    nb.frequency_class.toUpperCase()</span><br/>On-time Rate: ${
    (nb.on_time_rate * 100).toFixed(0)}%`;
    if (assignment) {
      popupContent += `<br/><hr style="margin:4px 0;"><b>Assigned to:</b> ${assignment.warehouse_name}<br/>
      ><b>Distance:</b> ${assignment.distance_km} km<br/><b>Delivery Cost:</b> ₹${assignment.delivery_cost}<br/>
      ><b>On-time Prob:</b> ${
      (assignment.on_time_prob * 100).toFixed(1)}%`;
      if (assignment.is_co_delivery) {
        popupContent += `<br/><span style="color:#16a34a;font-weight:700;">Co-delivery savings: ₹$
      {assignment.savings}/order</span>`;
      }
    }
    marker.bindPopup(popupContent);
  }
}, [neighbourhoods, warehouses, petrolPumps, assignments, routes, showOptimized, selectedWarehouseId]);

return <div ref={mapRef} className="map-container" />;
}

```

TypeScript src/main.tsx

React application entry point — 10 lines

```

import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)

```

Test src/optimization.test.ts

Unit tests (16 tests covering all optimization functions) — 136 lines

```

import { describe, it, expect } from 'vitest'
import {
  haversineDistance,
  classifyFrequency,
  classifyOrder,
  runOptimization,
  optimizeRoutes,
  computeDemandStatistics,
  defaultParams,
  vehicleRecommendations,
} from './optimization'
import type { Neighbourhood, Warehouse } from './types'

const mockNeighbourhoods: Neighbourhood[] = [
  { id: 'n1', name: 'Area A', lat: 15.35, lon: 75.10, population: 30000, daily_orders: 250, frequency_class: 'red', on_time_rate: 0.85, is_custom: false },
  { id: 'n2', name: 'Area B', lat: 15.36, lon: 75.12, population: 20000, daily_orders: 130, frequency_class: 'amber', on_time_rate: 0.88, is_custom: false },
  { id: 'n3', name: 'Area C', lat: 15.40, lon: 75.15, population: 15000, daily_orders: 80, frequency_class: 'green', on_time_rate: 0.92, is_custom: false },
]

const mockWarehouses: Warehouse[] = [
  { id: 'w1', name: 'WH Alpha', lat: 15.355, lon: 75.11, capacity_sqft: 10000, current_load_sqft: 5000, is_active: true, is_optimal: false, area_type: 'commercial' },
  { id: 'w2', name: 'WH Beta', lat: 15.395, lon: 75.14, capacity_sqft: 8000, current_load_sqft: 3000, is_active: true, is_optimal: false, area_type: 'industrial' },
]

describe('haversineDistance', () => {
  it('returns 0 for same point', () => {
    expect(haversineDistance(15.0, 75.0, 15.0, 75.0)).toBe(0)
  })

  it('calculates positive distance for different points', () => {
    const d = haversineDistance(15.35, 75.10, 15.40, 75.15)
    expect(d).toBeGreaterThan(0)
    expect(d).toBeLessThan(20) // should be under 20km for these coords
  })
})

describe('classifyFrequency', () => {
  it('classifies as red for >= 200 orders', () => {
    expect(classifyFrequency(200)).toBe('red')
    expect(classifyFrequency(300)).toBe('red')
  })

  it('classifies as amber for 120-199', () => {
    expect(classifyFrequency(120)).toBe('amber')
    expect(classifyFrequency(199)).toBe('amber')
  })

  it('classifies as green for < 120', () => {
    expect(classifyFrequency(50)).toBe('green')
    expect(classifyFrequency(119)).toBe('green')
  })
})

describe('classifyOrder', () => {
  it('classifies fragile when declared', () => {
    const result = classifyOrder(5, 2, true)
    expect(result.type).toBe('fragile')
    expect(result.vehicle.order_type).toBe('fragile')
  })

  it('classifies fragile for very light items', () => {
    const result = classifyOrder(1.5, 0.5, false)
    expect(result.type).toBe('fragile')
  })

  it('classifies sturdy for heavy items', () => {
    const result = classifyOrder(20, 4, false)
    expect(result.type).toBe('sturdy')
  })

  it('classifies normal for medium items', () => {

```

```

    const result = classifyOrder(5, 1.5, false)
    expect(result.type).toBe('normal')
  })
})

describe('runOptimization', () => {
  it('assigns all neighbourhoods within radius', () => {
    const params = defaultParams()
    const results = runOptimization(mockNeighbourhoods, mockWarehouses, params)
    expect(results.assignments.length).toBeGreaterThan(0)
    for (const a of results.assignments) {
      expect(a.distance_km).toBeLessThanOrEqual(params.service_radius_km)
    }
  })

  it('computes positive total cost', () => {
    const results = runOptimization(mockNeighbourhoods, mockWarehouses, defaultParams())
    expect(results.total_weighted_cost).toBeGreaterThan(0)
  })

  it('produces coverage <= 100', () => {
    const results = runOptimization(mockNeighbourhoods, mockWarehouses, defaultParams())
    expect(results.coverage_percent).toBeLessThanOrEqual(100)
  })
})

describe('optimizeRoutes', () => {
  it('generates routes for assigned warehouses', () => {
    const params = defaultParams()
    const results = runOptimization(mockNeighbourhoods, mockWarehouses, params)
    const routes = optimizeRoutes(results.assignments, mockWarehouses, mockNeighbourhoods, params)
    expect(routes.length).toBeGreaterThan(0)
    for (const r of routes) {
      expect(r.num_stops).toBeGreaterThan(0)
      expect(r.total_distance_km).toBeGreaterThan(0)
    }
  })
})

describe('computeDemandStatistics', () => {
  it('computes correct totals', () => {
    const stats = computeDemandStatistics(mockNeighbourhoods)
    expect(stats.totalOrders).toBe(460)
    expect(stats.totalPopulation).toBe(65000)
    expect(stats.frequencyDistribution.red).toBe(1)
    expect(stats.frequencyDistribution.amber).toBe(1)
    expect(stats.frequencyDistribution.green).toBe(1)
  })

  it('calculates peak day demand', () => {
    const stats = computeDemandStatistics(mockNeighbourhoods)
    expect(stats.peakDayDemand).toBe(Math.round(460 * 1.35))
  })
})

describe('vehicleRecommendations', () => {
  it('returns all three types', () => {
    const recs = vehicleRecommendations()
    expect(recs.length).toBe(3)
    const types = recs.map(r => r.order_type)
    expect(types).toContain('fragile')
    expect(types).toContain('normal')
    expect(types).toContain('sturdy')
  })
})

```

```

import { useState, useEffect, useCallback } from 'react';
import Papa from 'papaparse';
import MapView from './MapView';
import type {
  Neighbourhood,
  Warehouse,
  Order,
  PetrolPump,
  OptimizationParams,
  OptimizationResults,
  Assignment,
  DeliveryRoute,
  OptimizationRun,
} from './types';
import {
  fetchNeighbourhoods,
  fetchWarehouses,
  fetchOrders,
  fetchPetrolPumps,
  fetchOptimizationRuns,
  fetchRoutes,
  addNeighbourhood,
  addOrder,
  updateOrder,
  deleteNeighbourhood,
  deleteOrder,
  bulkInsertNeighbourhoods,
  saveOptimizationRun,
  saveRoutes,
  updateWarehouseOptimal,
} from './api';
import {
  runOptimization,
  optimizeRoutes,
  computeDemandStatistics,
  computeReliabilityMatrix,
  classifyOrder,
  vehicleRecommendations,
  sensitivityAnalysis,
  optimizeWarehouseSpace,
  defaultParams,
  haversineDistance,
  classifyFrequency,
} from './optimization';

type Tab = 'dashboard' | 'map' | 'data' | 'optimize' | 'routes' | 'orders' | 'analytics' | 'innovate';

export default function App() {
  const [tab, setTab] = useState<Tab>('dashboard');
  const [neighbourhoods, setNeighbourhoods] = useState<Neighbourhood[]>([]);
  const [warehouses, setWarehouses] = useState<Warehouse[]>([]);
  const [orders, setOrders] = useState<Order[]>([]);
  const [petrolPumps, setPetrolPumps] = useState<PetrolPump[]>([]);
  const [pastRuns, setPastRuns] = useState<OptimizationRun[]>([]);
  const [savedRoutes, setSavedRoutes] = useState<DeliveryRoute[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState('');

  const [params, setParams] = useState<OptimizationParams>(defaultParams());
  const [optimizationResults, setOptimizationResults] = useState<OptimizationResults | null>(null);
  const [optimizedRoutes, setOptimizedRoutes] = useState<DeliveryRoute[]>([]);
  const [showOptimized, setShowOptimized] = useState(false);
  const [selectedWarehouseId, setSelectedWarehouseId] = useState<string | null>(null);
  const [optimizing, setOptimizing] = useState(false);

  const loadData = useCallback(async () => {
    try {
      setLoading(true);
      const [nbs, whs, ords, pps, runs, rts] = await Promise.all([
        fetchNeighbourhoods(),
        fetchWarehouses(),
        fetchOrders(),
        fetchPetrolPumps(),
        fetchOptimizationRuns(),
        fetchRoutes(),
      ]);
    }
  });

```

```

    });
    setNeighbourhoods(nbs);
    setWarehouses(whs);
    setOrders(ords);
    setPetrolPumps(pps);
    setPastRuns(runs);
    setSavedRoutes(rts);
  } catch (e: any) {
    setError(`Failed to load data: ${e.message}`);
  } finally {
    setLoading(false);
  }
}, []);

useEffect(() => {
  loadData();
}, [loadData]);

const clearMessages = () => {
  setError('');
  setSuccess('');
};

const handleOptimize = async () => {
  setOptimizing(true);
  clearMessages();
  try {
    const results = runOptimization(neighbourhoods, warehouses, params);
    const routes = optimizeRoutes(results.assignments, warehouses, neighbourhoods, params);
    setOptimizationResults(results);
    setOptimizedRoutes(routes);
    setShowOptimized(true);

    const optimalIds = results.warehouse_costs.map((w) => w.warehouse_id);
    await updateWarehouseOptimal(optimalIds);
    await saveOptimizationRun(`Run ${new Date().toLocaleString()}`, params, results);
    await saveRoutes(routes);
    await loadData();
    setSuccess(
      `Optimization complete: ${results.warehouse_costs.length} warehouses selected, $
{results.assignments.length} neighbourhoods served, total cost ₹$
{results.total_weighted_cost.toLocaleString()}`
    );
  } catch (e: any) {
    setError(`Optimization failed: ${e.message}`);
  } finally {
    setOptimizing(false);
  }
};

const stats = computeDemandStatistics(neighbourhoods);
const reliability = optimizationResults
  ? null
  : computeReliabilityMatrix(neighbourhoods, warehouses, params.service_radius_km);

if (loading) {
  return (
    <div className="app">
      <div className="header">
        <h1>SAVEKART</h1>
      </div>
      <div className="loading">Loading Hubli-Dharwad city data...</div>
    </div>
  );
}

return (
  <div className="app">
    <div className="header">
      <div>
        <h1>SAVEKART</h1>
        <div className="subtitle">Warehouse Location Optimization Platform</div>
      </div>
      <div className="header-right">
        <span className="city-badge">Hubli-Dharwad, Karnataka</span>
      </div>
    </div>
  </div>
);

```

```

<div style={{ padding: '12px 24px', background: 'white', borderBottom: '1px solid var(--neutral-200)' }}
>
  <div className="tabs">
    {[
      ['dashboard', 'Dashboard'],
      ['map', 'Map View'],
      ['data', 'Data Upload'],
      ['optimize', 'Optimization Solver'],
      ['routes', 'Route Planner'],
      ['orders', 'Order Tracking'],
      ['analytics', 'Analytics & Sensitivity'],
      ['innovate', 'Innovate'],
    ] as [Tab, string][]}.map(([key, label]) => (
      <button
        key={key}
        className={`tab ${tab === key ? 'active' : ''}`}
        onClick={() => { setTab(key); clearMessages(); }}
      >
        {label}
      </button>
    ))
  </div>
</div>

<div className="main-content">
  {error && <div className="error-banner">{error}</div>}
  {success && <div className="success-banner">{success}</div>}

  {tab === 'dashboard' && (
    <DashboardView
      stats={stats}
      neighbourhoods={neighbourhoods}
      warehouses={warehouses}
      orders={orders}
      optimizationResults={optimizationResults}
      pastRuns={pastRuns}
      onOptimize={handleOptimize}
      optimizing={optimizing}
    />
  )}

  {tab === 'map' && (
    <MapViewTab
      neighbourhoods={neighbourhoods}
      warehouses={warehouses}
      petrolPumps={petrolPumps}
      assignments={optimizationResults?.assignments}
      routes={optimizedRoutes}
      showOptimized={showOptimized}
      selectedWarehouseId={selectedWarehouseId}
      onSelectWarehouse={setSelectedWarehouseId}
      onToggleOptimized={() => setShowOptimized(!showOptimized)}
    />
  )}

  {tab === 'data' && (
    <DataTab
      neighbourhoods={neighbourhoods}
      warehouses={warehouses}
      onAddNeighbourhood={async (nb) => {
        try {
          await addNeighbourhood(nb);
          await loadData();
          setSuccess(`Added neighbourhood: ${nb.name}`);
        } catch (e: any) { setError(e.message); }
      }}
      onDeleteNeighbourhood={async (id) => {
        try {
          await deleteNeighbourhood(id);
          await loadData();
          setSuccess('Neighbourhood deleted');
        } catch (e: any) { setError(e.message); }
      }}
      onBulkUpload={async (rows) => {
        try {
          const count = await bulkInsertNeighbourhoods(rows);

```

```

        await loadData();
        setSuccess(`Uploaded ${count} neighbourhoods from CSV`);
    } catch (e: any) { setError(e.message); }
    }}
    />
  )}

  {tab === 'optimize' && (
    <OptimizeTab
      params={params}
      setParams={setParams}
      onOptimize={handleOptimize}
      optimizing={optimizing}
      results={optimizationResults}
      neighbourhoods={neighbourhoods}
      warehouses={warehouses}
    />
  )}

  {tab === 'routes' && (
    <RoutesTab
      routes={optimizedRoutes.length > 0 ? optimizedRoutes : savedRoutes}
      warehouses={warehouses}
      petrolPumps={petrolPumps}
      neighbourhoods={neighbourhoods}
    />
  )}

  {tab === 'orders' && (
    <OrdersTab
      orders={orders}
      neighbourhoods={neighbourhoods}
      warehouses={warehouses}
      onAddOrder={async (order) => {
        try {
          await addOrder(order);
          await loadData();
          setSuccess(`Order ${order.order_ref} created`);
        } catch (e: any) { setError(e.message); }
      }}
      onUpdateOrder={async (id, updates) => {
        try {
          await updateOrder(id, updates);
          await loadData();
          setSuccess('Order updated');
        } catch (e: any) { setError(e.message); }
      }}
      onDeleteOrder={async (id) => {
        try {
          await deleteOrder(id);
          await loadData();
          setSuccess('Order deleted');
        } catch (e: any) { setError(e.message); }
      }}
    />
  )}

  {tab === 'analytics' && (
    <AnalyticsTab
      neighbourhoods={neighbourhoods}
      warehouses={warehouses}
      params={params}
      optimizationResults={optimizationResults}
    />
  )}

  {tab === 'innovate' && <InnovateTab />}
</div>
</div>
);
}

// ===== Dashboard =====
function DashboardView({
  stats,
  neighbourhoods,
  warehouses,

```

```

orders,
optimizationResults,
pastRuns,
onOptimize,
optimizing,
}: {
  stats: ReturnType<typeof computeDemandStatistics>;
  neighbourhoods: Neighbourhood[];
  warehouses: Warehouse[];
  orders: Order[];
  optimizationResults: OptimizationResults | null;
  pastRuns: OptimizationRun[];
  onOptimize: () => void;
  optimizing: boolean;
}) {
  return (
    <div>
      <div className="stats-grid">
        <div className="stat-card">
          <div className="stat-label">Total Neighbourhoods</div>
          <div className="stat-value">{neighbourhoods.length}</div>
          <div className="stat-sub">{stats.totalPopulation.toLocaleString()} population</div>
        </div>
        <div className="stat-card">
          <div className="stat-label">Daily Orders</div>
          <div className="stat-value">{stats.totalOrders.toLocaleString()}</div>
          <div className="stat-sub">Peak day: {stats.peakDayDemand.toLocaleString()}</div>
        </div>
        <div className="stat-card">
          <div className="stat-label">Warehouse Candidates</div>
          <div className="stat-value">{warehouses.length}</div>
          <div className="stat-sub">{warehouses.filter(w => w.is_optimal).length} currently optimal</div>
        </div>
        <div className="stat-card">
          <div className="stat-label">Tracked Orders</div>
          <div className="stat-value">{orders.length}</div>
          <div className="stat-sub">{orders.filter(o => o.status === 'delivered').length} delivered</div>
        </div>
        <div className="stat-card">
          <div className="stat-label">Red Zone Areas</div>
          <div className="stat-value" style={{ color: 'var(--error-500)' }}>{stats.frequencyDistribution.red}</div>
        </div>
        <div className="stat-sub">{stats.redOrders} daily orders</div>
      </div>
      <div className="stat-card">
        <div className="stat-label">Optimization Runs</div>
        <div className="stat-value">{pastRuns.length}</div>
        <div className="stat-sub">Last: {pastRuns[0]?.created_at ? new
Date(pastRuns[0].created_at).toLocaleDateString() : 'N/A'}</div>
      </div>
    </div>

    {optimizationResults && (
      <div className="card mb-20">
        <div className="card-title">Latest Optimization Summary</div>
        <div className="grid-4">
          <div>
            <div className="stat-label">Total Weighted Cost</div>
            <div className="stat-value">₹{optimizationResults.total_weighted_cost.toLocaleString()}</div>
          </div>
          <div>
            <div className="stat-label">Coverage</div>
            <div className="stat-value">{optimizationResults.coverage_percent}%</div>
          </div>
          <div>
            <div className="stat-label">Avg On-Time Probability</div>
            <div className="stat-value">{(optimizationResults.avg_on_time_rate * 100).toFixed(1)}%</div>
          </div>
          <div>
            <div className="stat-label">Co-Delivery Savings</div>
            <div className="stat-value" style={{ color: 'var(--secondary-600)' }}>₹
{optimizationResults.total_co_delivery_savings.toLocaleString()}</div>
          </div>
        </div>
      </div>
    )}
  )}

```

```

    <div className="grid-2">
      <div className="card">
        <div className="card-title">Demand Distribution by Frequency Class</div>
        <div style={{ display: 'flex', gap: '16px', flexWrap: 'wrap' }}>
          <FrequencyBar label="Red (High Frequency)" count={stats.frequencyDistribution.red}
orders={stats.redOrders} color="var(--error-500)" max={stats.totalOrders} />
          <FrequencyBar label="Amber (Medium)" count={stats.frequencyDistribution.amber}
orders={stats.amberOrders} color="var(--accent-500)" max={stats.totalOrders} />
          <FrequencyBar label="Green (Low)" count={stats.frequencyDistribution.green}
orders={stats.greenOrders} color="var(--secondary-500)" max={stats.totalOrders} />
        </div>
        <div className="mt-20">
          <div className="stat-label">Average Orders per Neighbourhood</div>
          <div style={{ fontSize: '1.2rem', fontWeight: 700 }}>{stats.avgOrdersPerNb} <span
style={{ fontSize: '0.8rem', color: 'var(--neutral-500)' }}>±{stats.stdDev}</span></div>
        </div>

        <div className="card">
          <div className="card-title">Quick Actions</div>
          <p style={{ fontSize: '0.85rem', color: 'var(--neutral-600)', marginBottom: '14px' }}>
            Run the optimization solver to determine optimal warehouse locations and neighbourhood assignments
for Hubli-Dharwad.
          </p>
          <button className="btn btn-primary" onClick={onOptimize} disabled={optimizing}>
            {optimizing ? 'Optimizing...' : 'Run Optimization'}
          </button>
          <div className="mt-20">
            <div className="stat-label">Top 5 High-Demand Neighbourhoods</div>
            <div className="scrollable-table mt-20" style={{ maxHeight: '200px' }}>
              <table>
                <thead>
                  <tr><th>Name</th><th>Orders</th><th>Class</th></tr>
                </thead>
                <tbody>
                  {neighbourhoods.slice(0, 5).map(nb => (
                    <tr key={nb.id}>
                      <td>{nb.name}</td>
                      <td>{nb.daily_orders}</td>
                      <td><span className={`badge badge-${nb.frequency_class}`}>{nb.frequency_class}</span></td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          </div>
        </div>
      </div>
    );
  }

function FrequencyBar({ label, count, orders, color, max }: { label: string; count: number; orders: number;
color: string; max: number }) {
  const pct = max > 0 ? (orders / max) * 100 : 0;
  return (
    <div style={{ flex: '1 1 140px' }}>
      <div style={{ fontSize: '0.78rem', fontWeight: 600, marginBottom: '4px' }}>{label}</div>
      <div style={{ fontSize: '0.72rem', color: 'var(--neutral-500)' }}>{count} areas, {orders} orders</div>
      <div className="progress-bar mt-20" style={{ marginTop: '6px' }}>
        <div className="progress-fill" style={{ width: `${pct}%`, background: color }} />
      </div>
    </div>
  );
}

// ===== Map View Tab =====
function MapViewTab({
  neighbourhoods,
  warehouses,
  petrolPumps,
  assignments,
  routes,
  showOptimized,
  selectedWarehouseId,
  onSelectWarehouse,

```

```

    onToggleOptimized,
  }: {
    neighbourhoods: Neighbourhood[];
    warehouses: Warehouse[];
    petrolPumps: PetrolPump[];
    assignments?: Assignment[];
    routes: DeliveryRoute[];
    showOptimized: boolean;
    selectedWarehouseId: string | null;
    onSelectWarehouse: (id: string | null) => void;
    onToggleOptimized: () => void;
  }) {
    return (
      <div>
        <div className="flex-between">
          <div>
            <div className="section-title">City Map – Hubli-Dharwad</div>
            <div className="section-desc">Visualize neighbourhoods, warehouses, petrol pumps, and optimized
assignments</div>
          </div>
          <div style={{ display: 'flex', alignItems: 'center', gap: '12px' }}>
            <span style={{ fontSize: '0.82rem', color: 'var(--neutral-600)' }}>Show Optimized</span>
            <div className={`toggle ${showOptimized ? 'active' : ''}`} onClick={onToggleOptimized}>
              <div className="toggle-thumb" />
            </div>
          </div>
        </div>

        <div className="legend mb-20">
          <div className="legend-item"><div className="legend-dot" style={{ background: 'var(--error-500)' }} />
Red Zone (≥200 orders/day)</div>
          <div className="legend-item"><div className="legend-dot" style={{ background: 'var(--accent-500)' }} />
> Amber Zone (120-199)</div>
          <div className="legend-item"><div className="legend-dot" style={{ background: 'var(--secondary-500)' }} /> Green Zone (<120)</div>
          <div className="legend-item"><div className="legend-dot" style={{ background: 'var(--primary-600)',
borderRadius: '4px' }} /> Warehouse</div>
          <div className="legend-item"><div className="legend-dot" style={{ background: 'var(--secondary-600)',
borderRadius: '4px' }} /> Optimal Warehouse</div>
          <div className="legend-item"><div className="legend-dot" style={{ background: 'var(--accent-600)',
borderRadius: '4px' }} /> Petrol Pump</div>
          <div className="legend-item"><div className="legend-line" style={{ background: 'var(--secondary-500)' }} /> Co-delivery link</div>
          <div className="legend-item"><div className="legend-line" style={{ background: 'var(--primary-500)' }} /> Route</div>
        </div>

        <MapView
          neighbourhoods={neighbourhoods}
          warehouses={warehouses}
          petrolPumps={petrolPumps}
          assignments={assignments}
          routes={routes}
          showOptimized={showOptimized}
          selectedWarehouseId={selectedWarehouseId}
          center={[15.39, 75.08]}
          zoom={12}
        />

        <div className="grid-2 mt-20">
          <div className="card">
            <div className="card-title">Warehouse Filter</div>
            <p style={{ fontSize: '0.82rem', color: 'var(--neutral-600)', marginBottom: '10px' }}>Click a
warehouse to highlight its service area on the map.</p>
            <div className="scrollable-table" style={{ maxHeight: '200px' }}>
              <table>
                <thead><tr><th>Name</th><th>Status</th><th>Action</th></tr></thead>
                <tbody>
                  {warehouses.map(wh => (
                    <tr key={wh.id}>
                      <td>{wh.name}</td>
                      <td>{wh.is_optimal ? <span className="badge badge-green">Optimal</span> : <span
className="badge badge-blue">Candidate</span>}</td>
                      <td><button className="btn btn-secondary" style={{ padding: '4px 10px', fontSize:
'0.75rem' }} onClick={() => onSelectWarehouse(selectedWarehouseId === wh.id ? null : wh.id)}
>{selectedWarehouseId === wh.id ? 'Selected' : 'Select'}</button></td>
                    </tr>
                  )

```

```

    ))}
  </tbody>
</table>
</div>
</div>
<div className="card">
  <div className="card-title">Map Statistics</div>
  <div className="grid-2">
    <div><div className="stat-label">Total Markers</div><div className="stat-value" style={{ fontSize:
'1.2rem' }}>{neighbourhoods.length + warehouses.length + petrolPumps.length}</div></div>
    <div><div className="stat-label">5km Coverage</div><div className="stat-value" style={{ fontSize:
'1.2rem' }}>{showOptimized && assignments ? `${assignments.length}/${neighbourhoods.length}` : 'Run
optimization'}</div></div>
  </div>
  <p style={{ fontSize: '0.78rem', color: 'var(--neutral-500)', marginTop: '10px' }}>
    Circle size represents daily order volume. Click any marker for detailed information.
  </p>
</div>
</div>
</div>
);
}

// ===== Data Upload Tab =====
function DataTab({
  neighbourhoods,
  warehouses,
  onAddNeighbourhood,
  onDeleteNeighbourhood,
  onBulkUpload,
}): {
  neighbourhoods: Neighbourhood[];
  warehouses: Warehouse[];
  onAddNeighbourhood: (nb: Omit<Neighbourhood, 'id' | 'created_at'>) => void;
  onDeleteNeighbourhood: (id: string) => void;
  onBulkUpload: (rows: { name: string; lat: number; lon: number; daily_orders: number }[]) => void;
}() {
  const [showAddForm, setShowAddForm] = useState(false);
  const [newNb, setNewNb] = useState({ name: '', lat: 15.35, lon: 75.1, population: 0, daily_orders: 50 });

  const handleFile = (file: File) => {
    Papa.parse(file, {
      header: true,
      complete: (results) => {
        const rows = results.data
          .filter((r: any) => r.name && r.lat && r.lon && r.daily_orders)
          .map((r: any) => ({
            name: String(r.name),
            lat: parseFloat(r.lat),
            lon: parseFloat(r.lon),
            daily_orders: parseInt(r.daily_orders) || 0,
          }));
        .filter((r) => !isNaN(r.lat) && !isNaN(r.lon));
        if (rows.length === 0) {
          alert('No valid rows found. CSV needs columns: name, lat, lon, daily_orders');
          return;
        }
        onBulkUpload(rows);
      },
      error: (err) => alert(`CSV parse error: ${err.message}`),
    });
  };

  const downloadTemplate = () => {
    const csv = 'name,lat,lon,daily_orders\nKeshwapur,15.3470,75.1370,280\nVidyanagar,15.3610,75.1480,240\n';
    const blob = new Blob([csv], { type: 'text/csv' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = 'savekart_neighbourhood_template.csv';
    a.click();
    URL.revokeObjectURL(url);
  };

  return (
    <div>
      <div className="section-title">Data Upload & Validation</div>

```

```

<div className="section-desc">Upload neighbourhood data via CSV (name, lat, lon, daily_orders) or add
entries manually</div>

<div className="grid-2">
  <div className="card">
    <div className="card-title">CSV Upload</div>
    <p style={{ fontSize: '0.85rem', color: 'var(--neutral-600)', marginBottom: '14px' }}>
      Your CSV needs only 4 columns: <b>name</b>, <b>lat</b>, <b>lon</b>, <b>daily_orders</b>. The
platform automatically classifies frequency and computes reliability.
    </p>
    <label className="file-upload" style={{ display: 'block' }}>
      <div className="file-upload-icon">&#128193;</div>
      <div style={{ fontWeight: 600, fontSize: '0.9rem' }}>Click to upload CSV file</div>
      <div style={{ fontSize: '0.78rem', color: 'var(--neutral-500)', marginTop: '4px' }}>Supported
format: .csv with headers</div>
      <input
        type="file"
        accept=".csv"
        style={{ display: 'none' }}
        onChange={(e) => { if (e.target.files?.[0]) handleFile(e.target.files[0]); }}
      />
    </label>
    <button className="btn btn-secondary mt-20" onClick={downloadTemplate}>Download CSV Template</
button>
  </div>

  <div className="card">
    <div className="card-title">Add Neighbourhood Manually</div>
    {!showAddForm ? (
      <button className="btn btn-primary" onClick={() => setShowAddForm(true)}>+ Add Neighbourhood</
button>
    ) : (
      <div>
        <div className="form-group">
          <label className="label">Name</label>
          <input className="input" value={newNb.name} onChange={(e) => setNewNb({ ...newNb, name:
e.target.value })} placeholder="e.g. Keshwapur" />
        </div>
        <div className="form-row">
          <div className="form-group">
            <label className="label">Latitude</label>
            <input className="input" type="number" step="0.0001" value={newNb.lat} onChange={(e) =>
setNewNb({ ...newNb, lat: parseFloat(e.target.value) })} />
          </div>
          <div className="form-group">
            <label className="label">Longitude</label>
            <input className="input" type="number" step="0.0001" value={newNb.lon} onChange={(e) =>
setNewNb({ ...newNb, lon: parseFloat(e.target.value) })} />
          </div>
        </div>
        <div className="form-row">
          <div className="form-group">
            <label className="label">Population</label>
            <input className="input" type="number" value={newNb.population} onChange={(e) =>
setNewNb({ ...newNb, population: parseInt(e.target.value) || 0 })} />
          </div>
          <div className="form-group">
            <label className="label">Daily Orders</label>
            <input className="input" type="number" value={newNb.daily_orders} onChange={(e) =>
setNewNb({ ...newNb, daily_orders: parseInt(e.target.value) || 0 })} />
          </div>
        </div>
        <div style={{ display: 'flex', gap: '10px' }}>
          <button className="btn btn-primary" onClick={() => {
            if (!newNb.name) return;
            onAddNeighbourhood({
              ...newNb,
              frequency_class: classifyFrequency(newNb.daily_orders),
              on_time_rate: 0.88,
              is_custom: true,
            });
            setShowAddForm(false);
            setNewNb({ name: '', lat: 15.35, lon: 75.1, population: 0, daily_orders: 50 });
          }}>Save</button>
          <button className="btn btn-secondary" onClick={() => setShowAddForm(false)}>Cancel</button>
        </div>
      </div>
    )
  </div>

```

```

    })
  </div>
</div>

<div className="card mt-20">
  <div className="flex-between">
    <div className="card-title">Neighbourhoods ({neighbourhoods.length})</div>
  </div>
  <div className="scrollable-table">
    <table>
      <thead>
        <tr><th>Name</th><th>Lat</th><th>Lon</th><th>Pop.</th><th>Orders</th><th>Class</th><th>On-Time</th><th>Source</th><th></th></tr>
      </thead>
      <tbody>
        {neighbourhoods.map(nb => (
          <tr key={nb.id}>
            <td>{nb.name}</td>
            <td>{nb.lat.toFixed(4)}</td>
            <td>{nb.lon.toFixed(4)}</td>
            <td>{nb.population.toLocaleString()}</td>
            <td>{nb.daily_orders}</td>
            <td><span className={`badge badge-${nb.frequency_class}`}>{nb.frequency_class}</span></td>
            <td>{(nb.on_time_rate * 100).toFixed(0)}%</td>
            <td>{nb.is_custom ? <span className="badge badge-blue">Custom</span> : <span
className="badge badge-green">Pre-loaded</span></td>
            <td>{nb.is_custom && <button className="btn btn-danger" style={{ padding: '3px 8px',
fontSize: '0.72rem' }} onClick={() => onDeleteNeighbourhood(nb.id)}>Delete</button></td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
</div>

<div className="card mt-20">
  <div className="card-title">Warehouses ({warehouses.length})</div>
  <div className="scrollable-table">
    <table>
      <thead>
        <tr><th>Name</th><th>Lat</th><th>Lon</th><th>Capacity (sq ft)</th><th>Load (sq ft)</th><th>Optimal</th><th>Type</th></tr>
      </thead>
      <tbody>
        {warehouses.map(wh => (
          <tr key={wh.id}>
            <td>{wh.name}</td>
            <td>{wh.lat.toFixed(4)}</td>
            <td>{wh.lon.toFixed(4)}</td>
            <td>{wh.capacity_sqft.toLocaleString()}</td>
            <td>{wh.current_load_sqft.toLocaleString()}</td>
            <td>{wh.is_optimal ? <span className="badge badge-green">Yes</span> : <span className="badge
badge-blue">No</span></td>
            <td>{wh.area_type}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
</div>
</div>
);
}

// ===== Optimize Tab =====
function OptimizeTab({
  params,
  setParams,
  onOptimize,
  optimizing,
  results,
  neighbourhoods,
  warehouses,
}): {
  params: OptimizationParams;
  setParams: (p: OptimizationParams) => void;
  onOptimize: () => void;

```

```

    optimizing: boolean;
    results: OptimizationResults | null;
    neighbourhoods: Neighbourhood[];
    warehouses: Warehouse[];
  }) {
    const spaceOpt = results
      ? optimizeWarehouseSpace(warehouses, results.assignments, 1.5)
      : [];

    return (
      <div>
        <div className="section-title">Warehouse Optimization Solver</div>
        <div className="section-desc">Configure parameters and run the linear programming optimization to
minimize weighted delivery cost</div>

        <div className="grid-2">
          <div className="card">
            <div className="card-title">Optimization Parameters</div>

            <div className="form-group">
              <label className="label">Max Warehouses: {params.max_warehouses}</label>
              <div className="slider-container">
                <input type="range" className="slider" min="1" max="8" value={params.max_warehouses}
onChange={(e) => setParams({ ...params, max_warehouses: parseInt(e.target.value) })} />
                <span className="slider-value">{params.max_warehouses}</span>
              </div>
            </div>

            <div className="form-group">
              <label className="label">Service Radius (km)</label>
              <div className="slider-container">
                <input type="range" className="slider" min="2" max="10" step="0.5"
value={params.service_radius_km} onChange={(e) => setParams({ ...params, service_radius_km:
parseFloat(e.target.value) })} />
                <span className="slider-value">{params.service_radius_km} km</span>
              </div>
            </div>

            <div className="form-group">
              <label className="label">Late Delivery Penalty (₹ per order)</label>
              <div className="slider-container">
                <input type="range" className="slider" min="0" max="200" step="5" value={params.late_penalty}
onChange={(e) => setParams({ ...params, late_penalty: parseInt(e.target.value) })} />
                <span className="slider-value">₹{params.late_penalty}</span>
              </div>
            </div>

            <div className="form-group">
              <label className="label">Time Expectation (hours)</label>
              <div className="slider-container">
                <input type="range" className="slider" min="2" max="72" step="2"
value={params.time_expectation_hours} onChange={(e) => setParams({ ...params, time_expectation_hours:
parseInt(e.target.value) })} />
                <span className="slider-value">{params.time_expectation_hours}h</span>
              </div>
            </div>

            <div className="form-group">
              <label className="label">Fuel Cost (₹/km)</label>
              <div className="slider-container">
                <input type="range" className="slider" min="5" max="15" step="0.5"
value={params.fuel_cost_per_km} onChange={(e) => setParams({ ...params, fuel_cost_per_km:
parseFloat(e.target.value) })} />
                <span className="slider-value">₹{params.fuel_cost_per_km}</span>
              </div>
            </div>

            <div className="form-group">
              <label className="label">Co-delivery Discount (%)</label>
              <div className="slider-container">
                <input type="range" className="slider" min="0" max="20" step="1"
value={params.co_delivery_discount_percent} onChange={(e) => setParams({ ...params,
co_delivery_discount_percent: parseInt(e.target.value) })} />
                <span className="slider-value">{params.co_delivery_discount_percent}%</span>
              </div>
            </div>
          </div>
        </div>
      </div>
    );
  }
}

```

```

<button className="btn btn-primary mt-20" onClick={onOptimize} disabled={optimizing} style={{ width:
'100%' }}>
  {optimizing ? 'Running Optimization...' : 'Run Optimization Solver'}
</button>
</div>

<div className="card">
  <div className="card-title">Optimization Results</div>
  {!results ? (
    <p style={{ color: 'var(--neutral-500)', fontSize: '0.85rem', padding: '30px 0', textAlign:
'center' }}>
      Run the optimization to see results here.
    </p>
  ) : (
    <div>
      <div className="grid-2 mb-20">
        <div><div className="stat-label">Total Weighted Cost</div><div className="stat-value">₹
{results.total_weighted_cost.toLocaleString()}</div></div>
        <div><div className="stat-label">Late Penalty Total</div><div className="stat-value">₹
{results.total_late_penalty.toLocaleString()}</div></div>
        <div><div className="stat-label">Fuel Cost Total</div><div className="stat-value">₹
{results.total_fuel_cost.toLocaleString()}</div></div>
        <div><div className="stat-label">Co-delivery Savings</div><div className="stat-value"
style={{ color: 'var(--secondary-600)' }}>₹{results.total_co_delivery_savings.toLocaleString()}</div></div>
        <div><div className="stat-label">Coverage</div><div className="stat-
value">{results.coverage_percent}%</div></div>
        <div><div className="stat-label">Avg On-Time Prob.</div><div className="stat-
value">{(results.avg_on_time_rate * 100).toFixed(1)}%</div></div>
      </div>

      <h4 style={{ marginBottom: '8px', fontSize: '0.9rem' }}>Selected Warehouses</h4>
      <div className="scrollable-table" style={{ maxHeight: '200px' }}>
        <table>
          <thead><tr><th>Warehouse</th><th>Areas Served</th><th>Orders</th><th>Cost</th><th>Load %</
th></tr></thead>
          <tbody>
            {results.warehouse_costs.map(wc => (
              <tr key={wc.warehouse_id}>
                <td>{wc.warehouse_name}</td>
                <td>{wc.neighbourhoods_served}</td>
                <td>{wc.total_orders}</td>
                <td>₹{wc.total_cost.toLocaleString()}</td>
                <td>{wc.load_utilization_percent}%</td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  )}
</div>
</div>

{results && (
  <div className="card mt-20">
    <div className="card-title">Neighbourhood-to-Warehouse Assignments</div>
    <div className="scrollable-table">
      <table>
        <thead><tr><th>Neighbourhood</th><th>Warehouse</th><th>Distance</th><th>Delivery Cost</
th><th>Late Penalty</th><th>On-Time Prob.</th><th>Orders</th><th>Co-Delivery</th><th>Savings</th></tr></thead>
        <tbody>
          {results.assignments.map(a => (
            <tr key={a.neighbourhood_id}>
              <td>{a.neighbourhood_name}</td>
              <td>{a.warehouse_name}</td>
              <td>{a.distance_km} km</td>
              <td>₹{a.delivery_cost}</td>
              <td>₹{a.late_penalty.toFixed(0)}</td>
              <td>{(a.on_time_prob * 100).toFixed(1)}%</td>
              <td>{a.orders}</td>
              <td>{a.is_co_delivery ? <span className="badge badge-green">Yes</span> : '-'}</td>
              <td>{a.savings > 0 ? `₹${a.savings}` : '-'}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>
)

```

```

    </div>
  </div>

  <div className="card mt-20">
    <div className="card-title">Warehouse Space Optimization</div>
    <div className="scrollable-table">
      <table>
        <thead><tr><th>Warehouse</th><th>Allocated Orders</th><th>Used Space (cft)</th><th>Capacity
(cft)</th><th>Utilization</th><th>Overflow</th></tr></thead>
        <tbody>
          {spaceOpt.map(s => (
            <tr key={s.warehouseId}>
              <td>{s.warehouseName}</td>
              <td>{s.allocatedOrders}</td>
              <td>{s.usedSpaceCft.toLocaleString()}</td>
              <td>{s.capacityCft.toLocaleString()}</td>
              <td>
                <div style={{ display: 'flex', alignItems: 'center', gap: '8px' }}>
                  <div className="progress-bar" style={{ width: '100px' }}>
                    <div className="progress-fill" style={{ width: `${s.utilizationPercent}%`,
background: s.utilizationPercent > 80 ? 'var(--error-500)' : 'var(--primary-500)' }} />
                  </div>
                  <span>{s.utilizationPercent}%</span>
                </div>
              </td>
              <td>{s.overflowOrders > 0 ? <span className="badge badge-red">{s.overflowOrders}
overflow</span> : <span className="badge badge-green">0K</span>}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>
</>
)}
</div>
);
}

// ===== Routes Tab =====
function RoutesTab({
  routes,
  warehouses,
  petrolPumps,
  neighbourhoods,
}): {
  routes: DeliveryRoute[];
  warehouses: Warehouse[];
  petrolPumps: PetrolPump[];
  neighbourhoods: Neighbourhood[];
}) {
  return (
    <div>
      <div className="section-title">Route Planner – Delivery Partner Routes</div>
      <div className="section-desc">Optimized routes with nearest-neighbour TSP, time windows, fuel costs, and
petrol pump indicators</div>

      <div className="grid-2">
        <div className="card">
          <div className="card-title">Optimized Routes ({routes.length})</div>
          {routes.length === 0 ? (
            <p style={{ color: 'var(--neutral-500)', fontSize: '0.85rem', padding: '20px 0', textAlign:
'center' }}>Run optimization to generate routes.</p>
          ) : (
            routes.map(route => {
              const wh = warehouses.find(w => w.id === route.warehouse_id);
              return (
                <div key={route.id} className="route-card">
                  <h4>{route.route_name}</h4>
                  <div className="route-meta">
                    <span>Distance: {route.total_distance_km} km</span>
                    <span>Time: {route.total_time_min} min</span>
                    <span>Stops: {route.num_stops}</span>
                    <span>Orders: {route.total_orders}</span>
                    <span>Fuel: ₹{route.fuel_cost}</span>
                    {route.co_delivery_savings > 0 && <span style={{ color: 'var(--secondary-600)',
fontWeight: 600 }}>Savings: ₹{route.co_delivery_savings}</span>}

```

```

        </div>
        <div style={{ fontSize: '0.75rem', color: 'var(--neutral-500)', marginTop: '4px' }}>Vehicle:
{route.vehicle_type} | Partner: {route.partner_name}</div>
        <div className="route-stops">
          <span className="route-stop-chip" style={{ background: 'var(--primary-100)', borderColor:
'var(--primary-300)' }}>{wh?.name ?? 'Warehouse'}</span>
          {route.stop_sequence.map((stop, i) => (
            <span key={stop.neighbourhood_id} style={{ display: 'flex', alignItems: 'center', gap:
'4px' }}>
              <span className="route-arrow">→</span>
              <span className="route-stop-chip">{stop.sequence}. {stop.name} ({stop.orders})</span>
            </span>
          ))}
          <span className="route-arrow">→</span>
          <span className="route-stop-chip" style={{ background: 'var(--primary-100)', borderColor:
'var(--primary-300)' }}>Return</span>
        </div>
      </div>
    );
  })
}
</div>

<div className="card">
  <div className="card-title">Petrol Pumps Along Routes</div>
  <p style={{ fontSize: '0.82rem', color: 'var(--neutral-600)', marginBottom: '12px' }}>
    {petrolPumps.length} petrol pumps mapped across Hubli-Dharwad for refueling stops during delivery
  </p>
  <div className="scrollable-table">
    <table>
      <thead><tr><th>Pump</th><th>Brand</th><th>Location</th></tr></thead>
      <tbody>
        {petrolPumps.map(pp => (
          <tr key={pp.id}>
            <td>{pp.name}</td>
            <td><span className="badge badge-amber">{pp.brand}</span></td>
            <td>{pp.lat.toFixed(4)}, {pp.lon.toFixed(4)}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>

  <div className="mt-20">
    <div className="card-title">Vehicle Recommendations by Order Type</div>
    {vehicleRecommendations().map(vr => (
      <div key={vr.order_type} className="route-card">
        <div style={{ display: 'flex', alignItems: 'center', gap: '8px', marginBottom: '4px' }}>
          <span className={`badge badge-${vr.order_type === 'fragile' ? 'red' : vr.order_type ===
'sturdy' ? 'amber' : 'green'}`}>{vr.order_type}</span>
          <strong style={{ fontSize: '0.85rem' }}>{vr.vehicle}</strong>
        </div>
        <div style={{ fontSize: '0.78rem', color: 'var(--neutral-600)' }}>{vr.reason}</div>
        <div className="route-meta mt-20" style={{ marginTop: '6px' }}>
          <span>Capacity: {vr.load_capacity_kg} kg</span>
          <span>Cost: ₹{vr.cost_per_km}/km</span>
          <span>{vr.fuel_efficiency_km_per_l > 0 && <span>Efficiency: {vr.fuel_efficiency_km_per_l} km/L</
span}>
        </div>
      </div>
    ))}
  </div>
</div>
</div>
);
}

// ===== Orders Tab =====
function OrdersTab({
  orders,
  neighbourhoods,
  warehouses,
  onAddOrder,
  onUpdateOrder,
  onDeleteOrder,

```

```

}: {
  orders: Order[];
  neighbourhoods: Neighbourhood[];
  warehouses: Warehouse[];
  onAddOrder: (order: Partial<Order>) => void;
  onUpdateOrder: (id: string, updates: Partial<Order>) => void;
  onDeleteOrder: (id: string) => void;
}) {
  const [showForm, setShowForm] = useState(false);
  const [newOrder, setNewOrder] = useState({
    order_ref: '',
    neighbourhood_id: '',
    warehouse_id: '',
    order_type: 'normal' as 'fragile' | 'normal' | 'sturdy',
    weight_kg: 1.0,
    volume_cft: 1.0,
    delivery_address: '',
    declared_fragile: false,
  });

  const handleSubmit = () => {
    if (!newOrder.order_ref || !newOrder.neighbourhood_id) return;
    const classification = classifyOrder(newOrder.weight_kg, newOrder.volume_cft, newOrder.declared_fragile);
    onAddOrder({
      order_ref: newOrder.order_ref,
      neighbourhood_id: newOrder.neighbourhood_id,
      warehouse_id: newOrder.warehouse_id || null,
      order_type: classification.type,
      status: 'pending',
      weight_kg: newOrder.weight_kg,
      volume_cft: newOrder.volume_cft,
      delivery_address: newOrder.delivery_address,
      vehicle_type: classification.vehicle.vehicle,
    });
    setShowForm(false);
    setNewOrder({ order_ref: '', neighbourhood_id: '', warehouse_id: '', order_type: 'normal', weight_kg: 1.0,
      volume_cft: 1.0, delivery_address: '', declared_fragile: false });
  };

  return (
    <div>
      <div className="flex-between">
        <div>
          <div className="section-title">Order Tracking</div>
          <div className="section-desc">Track and manage customer orders with automatic type classification
and vehicle recommendation</div>
        </div>
        <button className="btn btn-primary" onClick={() => setShowForm(!showForm)}>{showForm ? 'Cancel' : '+
New Order'}</button>
      </div>

      {showForm && (
        <div className="card mb-20">
          <div className="card-title">Create New Order</div>
          <div className="form-row">
            <div className="form-group">
              <label className="label">Order Reference</label>
              <input className="input" value={newOrder.order_ref} onChange={(e) => setNewOrder({ ...newOrder,
order_ref: e.target.value })} placeholder="e.g. SK-001" />
            </div>
            <div className="form-group">
              <label className="label">Neighbourhood</label>
              <select className="select" value={newOrder.neighbourhood_id} onChange={(e) =>
setNewOrder({ ...newOrder, neighbourhood_id: e.target.value })}>
                <option value="">Select area</option>
                {neighbourhoods.map(nb => <option key={nb.id} value={nb.id}>{nb.name}</option>)}
              </select>
            </div>
          </div>
          <div className="form-row">
            <div className="form-group">
              <label className="label">Warehouse (optional)</label>
              <select className="select" value={newOrder.warehouse_id} onChange={(e) =>
setNewOrder({ ...newOrder, warehouse_id: e.target.value })}>
                <option value="">Auto-assign</option>
                {warehouses.map(wh => <option key={wh.id} value={wh.id}>{wh.name}</option>)}
              </select>
            </div>
          </div>
        </div>
      )}
    </div>
  );
}

```

```

    </div>
    <div className="form-group">
      <label className="label">Weight (kg)</label>
      <input className="input" type="number" step="0.1" value={newOrder.weight_kg} onChange={(e) =>
setNewOrder({ ...newOrder, weight_kg: parseFloat(e.target.value) })} />
    </div>
    <div className="form-group">
      <label className="label">Volume (cft)</label>
      <input className="input" type="number" step="0.1" value={newOrder.volume_cft} onChange={(e) =>
setNewOrder({ ...newOrder, volume_cft: parseFloat(e.target.value) })} />
    </div>
  </div>
  <div className="form-row">
    <div className="form-group">
      <label className="label">Delivery Address</label>
      <input className="input" value={newOrder.delivery_address} onChange={(e) =>
setNewOrder({ ...newOrder, delivery_address: e.target.value })} placeholder="Street address" />
    </div>
    <div className="form-group">
      <label className="label">Declared Fragile?</label>
      <select className="select" value={newOrder.declared_fragile ? 'yes' : 'no'} onChange={(e) =>
setNewOrder({ ...newOrder, declared_fragile: e.target.value === 'yes' })}>
        <option value="no">No</option>
        <option value="yes">Yes</option>
      </select>
    </div>
  </div>
  <div>
    {(() => {
      const cls = classifyOrder(newOrder.weight_kg, newOrder.volume_cft, newOrder.declared_fragile);
      return (
        <div style={{ fontSize: '0.82rem', padding: '10px 14px', background: 'var(--neutral-50)',
borderRadius: 'var(--radius-sm)', marginBottom: '12px' }}>
          Auto-classified as <strong>{cls.type}</strong> – Recommended vehicle:
          <strong>{cls.vehicle.vehicle}</strong>
        </div>
      );
    })()}
    <button className="btn btn-primary" onClick={handleSubmit}>Create Order</button>
  </div>
)}

<div className="card">
  <div className="card-title">All Orders ({orders.length})</div>
  {orders.length === 0 ? (
    <p style={{ color: 'var(--neutral-500)', fontSize: '0.85rem', padding: '20px 0', textAlign:
'center' }}>No orders yet. Create one to get started.</p>
  ) : (
    <div className="scrollable-table">
      <table>
        <thead>
          <tr><th>Ref</th><th>Area</th><th>Warehouse</th><th>Type</th><th>Status</th><th>Vehicle</
th><th>On Time</th><th>Savings</th><th>Actions</th></tr>
        </thead>
        <tbody>
          {orders.map(ord => {
            const nb = neighbourhoods.find(n => n.id === ord.neighbourhood_id);
            const wh = warehouses.find(w => w.id === ord.warehouse_id);
            return (
              <tr key={ord.id}>
                <td>{ord.order_ref}</td>
                <td>{nb?.name ?? '-'}</td>
                <td>{wh?.name ?? 'Unassigned'}</td>
                <td><span className={`badge badge-${ord.order_type === 'fragile' ? 'red' :
ord.order_type === 'sturdy' ? 'amber' : 'green'}`}>{ord.order_type}</span></td>
                <td>
                  <select
                    className="select"
                    style={{ width: 'auto', padding: '2px 6px', fontSize: '0.75rem' }}
                    value={ord.status}
                    onChange={(e) => onUpdateOrder(ord.id, { status: e.target.value as
Order['status'] })}
                  >
                    <option value="pending">Pending</option>
                    <option value="assigned">Assigned</option>
                    <option value="out_for_delivery">Out for Delivery</option>
                    <option value="delivered">Delivered</option>
                    <option value="failed">Failed</option>

```

```

        </select>
      </td>
      <td>{ord.vehicle_type ?? '-'}</td>
      <td>{ord.on_time === null ? '-' : ord.on_time ? <span className="badge badge-
green">Yes</span> : <span className="badge badge-red">Late</span>}</td>
      <td>{ord.co_delivery_savings > 0 ? `₹${ord.co_delivery_savings}` : '-'}</td>
      <td><button className="btn btn-danger" style={{ padding: '3px 8px', fontSize:
'0.72rem' }} onClick={() => onDeleteOrder(ord.id)}>Delete</button></td>
    </tr>
  </tbody>
</table>
</div>
)}
</div>
</div>
);
}

// ===== Analytics Tab =====
function AnalyticsTab({
  neighbourhoods,
  warehouses,
  params,
  optimizationResults,
}) {
  neighbourhoods: Neighbourhood[];
  warehouses: Warehouse[];
  params: OptimizationParams;
  optimizationResults: OptimizationResults | null;
}) {
  const [sensitivity, setSensitivity] = useState<ReturnType<typeof sensitivityAnalysis>>([]);

  useEffect(() => {
    setSensitivity(sensitivityAnalysis(neighbourhoods, warehouses, params));
  }, [neighbourhoods, warehouses, params]);

  const reliabilityMatrix = computeReliabilityMatrix(neighbourhoods, warehouses, params.service_radius_km);
  const stats = computeDemandStatistics(neighbourhoods);

  return (
    <div>
      <div className="section-title">Analytics & Sensitivity Analysis</div>
      <div className="section-desc">Demand statistics, reliability matrices, and sensitivity checks across
varying parameters</div>

      <div className="stats-grid">
        <div className="stat-card"><div className="stat-label">Total Daily Orders</div><div className="stat-
value">{stats.totalOrders}</div></div>
        <div className="stat-card"><div className="stat-label">Peak Day Demand</div><div className="stat-
value">{stats.peakDayDemand}</div><div className="stat-sub">x{stats.peakDayMultiplier} multiplier</div></div>
        <div className="stat-card"><div className="stat-label">Avg Orders/Area</div><div className="stat-
value">{stats.avgOrdersPerNb}</div><div className="stat-sub"> $\sigma = \{stats.stdDev\}$ </div></div>
        <div className="stat-card"><div className="stat-label">Max Orders</div><div className="stat-
value">{stats.maxOrders}</div></div>
        <div className="stat-card"><div className="stat-label">Min Orders</div><div className="stat-
value">{stats.minOrders}</div></div>
      </div>

      <div className="grid-2">
        <div className="card">
          <div className="card-title">Sensitivity Analysis</div>
          <p style={{ fontSize: '0.82rem', color: 'var(--neutral-600)', marginBottom: '12px' }}>How total cost
and coverage respond to changes in service radius, late penalty, and warehouse count.</p>
          <div className="scrollable-table" style={{ maxHeight: '350px' }}>
            <table>
              <thead>
                <tr>
                  <th>Scenario</th>
                  <th>Total Cost (₹)</th>
                  <th>Coverage</th>
                  <th>Warehouses</th>
                </tr>
              </thead>
              <tbody>
                {sensitivity.map((s, i) => (
                  <tr key={i}>
                    <td>{s.label}</td>
                    <td>₹{s.totalCost.toLocaleString()}</td>
                    <td>{s.coverage}%</td>
                    <td>{s.warehousesUsed}</td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </div>
      </div>
    </div>
  );
}

```

```

    ))}
  </tbody>
</table>
</div>
</div>

<div className="card">
  <div className="card-title">Reliability Matrix (Top 20)</div>
  <p style={{ fontSize: '0.82rem', color: 'var(--neutral-600)', marginBottom: '12px' }}>On-time
probability:  $P = \text{base\_rate} \times e^{(-\text{distance}/\text{radius})}$ </p>
  <div className="scrollable-table" style={{ maxHeight: '350px' }}>
    <table>
      <thead><tr><th>Neighbourhood</th><th>Warehouse</th><th>Distance</th><th>P(On-Time)</th></tr></thead>
      <tbody>
        {reliabilityMatrix.slice(0, 20).map((r, i) => (
          <tr key={i}>
            <td>{r.neighbourhood}</td>
            <td>{r.warehouse}</td>
            <td>{r.distance} km</td>
            <td>{(r.onTimeProb * 100).toFixed(1)}%</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
</div>
</div>

{optimizationResults && (
  <div className="card mt-20">
    <div className="card-title">Cost Breakdown</div>
    <div className="grid-3">
      <div>
        <div className="stat-label">Fuel Cost</div>
        <div className="stat-value">₹{optimizationResults.total_fuel_cost.toLocaleString()}</div>
        <div className="progress-bar mt-20" style={{ marginTop: '6px' }}>
          <div className="progress-fill" style={{ width: `${(optimizationResults.total_fuel_cost /
optimizationResults.total_weighted_cost) * 100}%`, background: 'var(--primary-500)' }} />
        </div>
      </div>
      <div>
        <div className="stat-label">Late Penalty</div>
        <div className="stat-value">₹{optimizationResults.total_late_penalty.toLocaleString()}</div>
        <div className="progress-bar mt-20" style={{ marginTop: '6px' }}>
          <div className="progress-fill" style={{ width: `${(optimizationResults.total_late_penalty /
optimizationResults.total_weighted_cost) * 100}%`, background: 'var(--error-500)' }} />
        </div>
      </div>
      <div>
        <div className="stat-label">Co-delivery Savings</div>
        <div className="stat-value" style={{ color: 'var(--secondary-600)' }}>₹
{optimizationResults.total_co_delivery_savings.toLocaleString()}</div>
        <div className="progress-bar mt-20" style={{ marginTop: '6px' }}>
          <div className="progress-fill" style={{ width: '100%', background: 'var(--secondary-500)' }} />
        </div>
      </div>
    </div>
  </div>
)
);
}

// ===== Innovate Tab =====
function InnovateTab() {
  return (
    <div>
      <div className="section-title">Innovation Lab</div>
      <div className="section-desc">SAVeKART is designed to be extensible – add your own innovative features
even after the platform is built</div>

      <div className="card mb-20" style={{ background: 'linear-gradient(135deg, var(--primary-50), var(--
secondary-50))', border: '1px solid var(--primary-200)' }}>
        <h3 style={{ marginBottom: '10px' }}>Add Your Own Feature</h3>
        <p style={{ fontSize: '0.88rem', color: 'var(--neutral-700)', marginBottom: '14px' }}>

```

SAVEKART's modular architecture allows new features to be plugged in without disrupting existing functionality. The optimization engine, data layer, and UI components are decoupled, so you can add new analysis modules, visualizations, or integrations at any time.

```
</p>
<div style={{ fontSize: '0.82rem', color: 'var(--neutral-600)' }}>
  <strong>To add a new feature:</strong>
  <ol style={{ paddingLeft: '20px', marginTop: '6px', lineHeight: 1.8 }}>
    <li>Add the mathematical model to <code style={{ background: 'var(--neutral-100)', padding: '1px 5px', borderRadius: '3px' }}>optimization.ts</code></li>
    <li>Create a new tab component in <code style={{ background: 'var(--neutral-100)', padding: '1px 5px', borderRadius: '3px' }}>App.tsx</code></li>
    <li>Add any new database tables via Supabase migrations</li>
    <li>Wire up the API calls in <code style={{ background: 'var(--neutral-100)', padding: '1px 5px', borderRadius: '3px' }}>api.ts</code></li>
  </ol>
</div>
</div>

<div className="grid-3">
  <div className="feature-card">
    <div className="feature-icon" style={{ background: 'var(--secondary-100)' }}>&#9851;</div>
    <div className="feature-title">Carbon Footprint Tracker</div>
    <div className="feature-desc">Calculate CO2 emissions per delivery route and suggest the greenest vehicle options. Track sustainability metrics over time.</div>
  </div>
  <div className="feature-card">
    <div className="feature-icon" style={{ background: 'var(--accent-100)' }}>&#128202;</div>
    <div className="feature-title">Demand Forecasting</div>
    <div className="feature-desc">Predict future order volumes using time-series analysis. Proactively adjust warehouse staffing and inventory.</div>
  </div>
  <div className="feature-card">
    <div className="feature-icon" style={{ background: 'var(--primary-100)' }}>&#129518;</div>
    <div className="feature-title">Crowdsourced Delivery</div>
    <div className="feature-desc">Allow community members to become delivery partners during peak hours, reducing cost and improving delivery times.</div>
  </div>
  <div className="feature-card">
    <div className="feature-icon" style={{ background: '#fee2e2' }}>&#128737;</div>
    <div className="feature-title">Weather-Aware Routing</div>
    <div className="feature-desc">Integrate weather forecasts to proactively reroute deliveries away from flooded or congested areas during monsoon.</div>
  </div>
  <div className="feature-card">
    <div className="feature-icon" style={{ background: 'var(--secondary-100)' }}>&#128176;</div>
    <div className="feature-title">Dynamic Pricing Engine</div>
    <div className="feature-desc">Adjust delivery charges based on demand, distance, and co-delivery availability. Pass savings directly to customers.</div>
  </div>
  <div className="feature-card">
    <div className="feature-icon" style={{ background: 'var(--accent-100)' }}>&#129682;</div>
    <div className="feature-title">Micro-Warehouse Network</div>
    <div className="feature-desc">Identify underutilized commercial spaces that can serve as pop-up micro-warehouses during peak demand periods.</div>
  </div>
</div>

<div className="card mt-20">
  <div className="card-title">Current Feature Set</div>
  <div className="grid-2">
    <div>
      <h4 style={{ fontSize: '0.88rem', marginBottom: '8px' }}>Optimization</h4>
      <ul style={{ paddingLeft: '20px', fontSize: '0.82rem', color: 'var(--neutral-600)', lineHeight: 1.8 }}>
        <li>Order-weighted delivery cost minimization (LP)</li>
        <li>Late-delivery penalty function</li>
        <li>5km service radius constraint</li>
        <li>Facility-location greedy heuristic</li>
        <li>Sensitivity analysis (radius, penalty, warehouse count)</li>
      </ul>
    </div>
    <div>
      <h4 style={{ fontSize: '0.88rem', marginBottom: '8px' }}>Logistics</h4>
      <ul style={{ paddingLeft: '20px', fontSize: '0.82rem', color: 'var(--neutral-600)', lineHeight: 1.8 }}>
        <li>Route optimization (nearest-neighbour TSP)</li>
        <li>Co-delivery detection and savings calculation</li>
      </ul>
    </div>
  </div>
</div>
```

```
        <li>Order classification: fragile / normal / sturdy</li>
        <li>Vehicle recommendation by order type</li>
        <li>Warehouse space optimization (bin-packing)</li>
        <li>Petrol pump indicators on routes</li>
    </ul>
</div>
</div>
</div>
</div>
);
}
```

CSS src/index.css

Global stylesheet with design system — 723 lines

```

:root {
  --primary-50: #eef7ff;
  --primary-100: #d9ecff;
  --primary-200: #bcdcff;
  --primary-300: #8ec5ff;
  --primary-400: #59a5ff;
  --primary-500: #3380f6;
  --primary-600: #1e63e8;
  --primary-700: #184dd1;
  --primary-800: #1a40a8;
  --primary-900: #1b3a85;
  --secondary-50: #f0fdf4;
  --secondary-100: #dcfce7;
  --secondary-200: #bbf7d0;
  --secondary-300: #86efac;
  --secondary-400: #4ade80;
  --secondary-500: #22c55e;
  --secondary-600: #16a34a;
  --secondary-700: #15803d;
  --accent-50: #fffbeb;
  --accent-100: #fef3c7;
  --accent-200: #fde68a;
  --accent-300: #fcd34d;
  --accent-400: #fbbf24;
  --accent-500: #f59e0b;
  --accent-600: #d97706;
  --success-500: #22c55e;
  --warning-500: #f59e0b;
  --error-500: #ef4444;
  --error-600: #dc2626;
  --neutral-50: #f8fafc;
  --neutral-100: #f1f5f9;
  --neutral-200: #e2e8f0;
  --neutral-300: #cbd5e1;
  --neutral-400: #94a3b8;
  --neutral-500: #64748b;
  --neutral-600: #475569;
  --neutral-700: #334155;
  --neutral-800: #1e293b;
  --neutral-900: #0f172a;
  --radius: 12px;
  --radius-sm: 8px;
  --shadow: 0 1px 3px rgba(0,0,0,0.08), 0 1px 2px rgba(0,0,0,0.06);
  --shadow-lg: 0 10px 30px rgba(0,0,0,0.12), 0 4px 12px rgba(0,0,0,0.08);
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  background: var(--neutral-50);
  color: var(--neutral-800);
  line-height: 1.5;
}

h1, h2, h3, h4 {
  line-height: 1.2;
  font-weight: 700;
  color: var(--neutral-900);
}

.app {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.header {
  background: linear-gradient(135deg, var(--primary-700), var(--primary-500));
  color: white;
  padding: 16px 24px;
  display: flex;

```

```
    align-items: center;
    justify-content: space-between;
    box-shadow: var(--shadow);
    position: sticky;
    top: 0;
    z-index: 1000;
}

.header h1 {
    color: white;
    font-size: 1.5rem;
    display: flex;
    align-items: center;
    gap: 10px;
}

.header .subtitle {
    font-size: 0.8rem;
    opacity: 0.85;
    font-weight: 400;
}

.header-right {
    display: flex;
    align-items: center;
    gap: 16px;
}

.city-badge {
    background: rgba(255,255,255,0.15);
    border: 1px solid rgba(255,255,255,0.3);
    border-radius: 20px;
    padding: 4px 14px;
    font-size: 0.8rem;
    font-weight: 500;
}

.tabs {
    display: flex;
    gap: 2px;
    background: var(--neutral-100);
    padding: 4px;
    border-radius: var(--radius);
    overflow-x: auto;
}

.tab {
    padding: 8px 18px;
    border: none;
    background: transparent;
    color: var(--neutral-600);
    font-weight: 600;
    font-size: 0.85rem;
    border-radius: var(--radius-sm);
    cursor: pointer;
    transition: all 0.2s;
    white-space: nowrap;
}

.tab:hover {
    background: var(--neutral-200);
    color: var(--neutral-800);
}

.tab.active {
    background: white;
    color: var(--primary-700);
    box-shadow: var(--shadow);
}

.main-content {
    flex: 1;
    padding: 24px;
    max-width: 1600px;
    margin: 0 auto;
    width: 100%;
}
```

```
.card {
  background: white;
  border-radius: var(--radius);
  padding: 20px;
  box-shadow: var(--shadow);
  border: 1px solid var(--neutral-200);
}

.card-title {
  font-size: 1rem;
  font-weight: 700;
  margin-bottom: 14px;
  color: var(--neutral-900);
  display: flex;
  align-items: center;
  gap: 8px;
}

.grid-2 {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
}

.grid-3 {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}

.grid-4 {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 16px;
}

.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(180px, 1fr));
  gap: 14px;
  margin-bottom: 20px;
}

.stat-card {
  background: white;
  border-radius: var(--radius);
  padding: 16px 18px;
  box-shadow: var(--shadow);
  border: 1px solid var(--neutral-200);
  transition: transform 0.2s, box-shadow 0.2s;
}

.stat-card:hover {
  transform: translateY(-2px);
  box-shadow: var(--shadow-lg);
}

.stat-label {
  font-size: 0.75rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--neutral-500);
  font-weight: 600;
}

.stat-value {
  font-size: 1.6rem;
  font-weight: 700;
  color: var(--primary-700);
  margin-top: 4px;
}

.stat-sub {
  font-size: 0.75rem;
  color: var(--neutral-500);
  margin-top: 2px;
}
```

```
}

.map-container {
  border-radius: var(--radius);
  overflow: hidden;
  box-shadow: var(--shadow);
  border: 1px solid var(--neutral-200);
  height: 520px;
}

.map-container .leaflet-container {
  height: 100%;
  width: 100%;
}

.btn {
  padding: 10px 20px;
  border: none;
  border-radius: var(--radius-sm);
  font-weight: 600;
  font-size: 0.85rem;
  cursor: pointer;
  transition: all 0.2s;
  display: inline-flex;
  align-items: center;
  gap: 6px;
}

.btn-primary {
  background: var(--primary-600);
  color: white;
}

.btn-primary:hover {
  background: var(--primary-700);
  transform: translateY(-1px);
  box-shadow: var(--shadow-lg);
}

.btn-secondary {
  background: var(--neutral-200);
  color: var(--neutral-700);
}

.btn-secondary:hover {
  background: var(--neutral-300);
}

.btn-success {
  background: var(--secondary-600);
  color: white;
}

.btn-success:hover {
  background: var(--secondary-700);
}

.btn-danger {
  background: var(--error-500);
  color: white;
}

.btn:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.input, .select {
  width: 100%;
  padding: 9px 12px;
  border: 1px solid var(--neutral-300);
  border-radius: var(--radius-sm);
  font-size: 0.85rem;
  background: white;
  transition: border-color 0.2s, box-shadow 0.2s;
}
```

```
.input:focus, .select:focus {
  outline: none;
  border-color: var(--primary-500);
  box-shadow: 0 0 0 3px var(--primary-100);
}

.label {
  font-size: 0.8rem;
  font-weight: 600;
  color: var(--neutral-700);
  margin-bottom: 4px;
  display: block;
}

.form-group {
  margin-bottom: 14px;
}

.form-row {
  display: flex;
  gap: 12px;
}

.form-row > * {
  flex: 1;
}

table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.82rem;
}

th {
  text-align: left;
  padding: 10px 12px;
  background: var(--neutral-50);
  color: var(--neutral-600);
  font-weight: 600;
  border-bottom: 2px solid var(--neutral-200);
  text-transform: uppercase;
  font-size: 0.72rem;
  letter-spacing: 0.03em;
}

td {
  padding: 10px 12px;
  border-bottom: 1px solid var(--neutral-100);
  color: var(--neutral-700);
}

tr:hover td {
  background: var(--neutral-50);
}

.badge {
  display: inline-block;
  padding: 2px 10px;
  border-radius: 20px;
  font-size: 0.72rem;
  font-weight: 600;
}

.badge-red {
  background: #fee2e2;
  color: #991b1b;
}

.badge-amber {
  background: #fef3c7;
  color: #92400e;
}

.badge-green {
  background: #dcfce7;
  color: #166534;
}
```

```
.badge-blue {
  background: var(--primary-100);
  color: var(--primary-700);
}

.badge-purple {
  background: #f3e8ff;
  color: #6b21a8;
}

.loading {
  text-align: center;
  padding: 40px;
  color: var(--neutral-500);
}

.error-banner {
  background: #fee2e2;
  color: #991b1b;
  padding: 12px 18px;
  border-radius: var(--radius-sm);
  margin-bottom: 16px;
  font-size: 0.85rem;
  font-weight: 500;
}

.success-banner {
  background: #dcfce7;
  color: #166534;
  padding: 12px 18px;
  border-radius: var(--radius-sm);
  margin-bottom: 16px;
  font-size: 0.85rem;
  font-weight: 500;
}

.file-upload {
  border: 2px dashed var(--neutral-300);
  border-radius: var(--radius);
  padding: 30px;
  text-align: center;
  cursor: pointer;
  transition: all 0.2s;
  background: var(--neutral-50);
}

.file-upload:hover {
  border-color: var(--primary-400);
  background: var(--primary-50);
}

.file-upload-icon {
  font-size: 2rem;
  color: var(--neutral-400);
  margin-bottom: 8px;
}

.progress-bar {
  height: 8px;
  background: var(--neutral-200);
  border-radius: 4px;
  overflow: hidden;
}

.progress-fill {
  height: 100%;
  background: var(--primary-500);
  transition: width 0.3s ease;
}

.section-title {
  font-size: 1.3rem;
  font-weight: 700;
  margin-bottom: 4px;
  color: var(--neutral-900);
}
```

```
.section-desc {
  font-size: 0.85rem;
  color: var(--neutral-500);
  margin-bottom: 20px;
}

.flex-between {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 16px;
}

.mt-20 { margin-top: 20px; }
.mb-20 { margin-bottom: 20px; }
.mb-10 { margin-bottom: 10px; }
.gap-10 { gap: 10px; }
.flex { display: flex; }
.flex-col { flex-direction: column; }
.flex-1 { flex: 1; }

.scrollable-table {
  max-height: 400px;
  overflow-y: auto;
  border-radius: var(--radius-sm);
  border: 1px solid var(--neutral-200);
}

.scrollable-table table {
  margin: 0;
}

.scrollable-table thead {
  position: sticky;
  top: 0;
  z-index: 10;
}

.legend {
  display: flex;
  gap: 16px;
  flex-wrap: wrap;
  padding: 10px 14px;
  background: white;
  border-radius: var(--radius-sm);
  border: 1px solid var(--neutral-200);
  font-size: 0.78rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 6px;
  color: var(--neutral-600);
}

.legend-dot {
  width: 12px;
  height: 12px;
  border-radius: 50%;
}

.legend-line {
  width: 20px;
  height: 3px;
  border-radius: 2px;
}

.route-card {
  background: var(--neutral-50);
  border: 1px solid var(--neutral-200);
  border-radius: var(--radius-sm);
  padding: 14px;
  margin-bottom: 10px;
}
```

```
.route-card h4 {
  font-size: 0.9rem;
  margin-bottom: 6px;
}

.route-meta {
  display: flex;
  gap: 16px;
  font-size: 0.78rem;
  color: var(--neutral-600);
  flex-wrap: wrap;
}

.route-stops {
  display: flex;
  align-items: center;
  gap: 4px;
  margin-top: 8px;
  flex-wrap: wrap;
  font-size: 0.75rem;
}

.route-stop-chip {
  background: white;
  border: 1px solid var(--neutral-200);
  border-radius: 16px;
  padding: 2px 10px;
  color: var(--neutral-600);
}

.route-arrow {
  color: var(--neutral-400);
}

.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0,0,0,0.4);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 2000;
  padding: 20px;
}

.modal {
  background: white;
  border-radius: var(--radius);
  padding: 24px;
  max-width: 540px;
  width: 100%;
  max-height: 85vh;
  overflow-y: auto;
  box-shadow: var(--shadow-lg);
}

.modal h3 {
  margin-bottom: 16px;
}

.slider-container {
  display: flex;
  align-items: center;
  gap: 10px;
}

.slider {
  flex: 1;
  -webkit-appearance: none;
  appearance: none;
  height: 6px;
  background: var(--neutral-200);
  border-radius: 3px;
  outline: none;
}
```

```
}

.slider::-webkit-slider-thumb {
  -webkit-appearance: none;
  appearance: none;
  width: 18px;
  height: 18px;
  background: var(--primary-600);
  border-radius: 50%;
  cursor: pointer;
  transition: transform 0.15s;
}

.slider::-webkit-slider-thumb:hover {
  transform: scale(1.15);
}

.slider-value {
  font-weight: 700;
  color: var(--primary-700);
  min-width: 50px;
  text-align: right;
  font-size: 0.85rem;
}

.feature-card {
  background: white;
  border-radius: var(--radius);
  padding: 18px;
  box-shadow: var(--shadow);
  border: 1px solid var(--neutral-200);
  transition: transform 0.2s, box-shadow 0.2s;
}

.feature-card:hover {
  transform: translateY(-3px);
  box-shadow: var(--shadow-lg);
}

.feature-icon {
  width: 40px;
  height: 40px;
  border-radius: 10px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.2rem;
  margin-bottom: 10px;
}

.feature-title {
  font-size: 0.9rem;
  font-weight: 700;
  margin-bottom: 4px;
}

.feature-desc {
  font-size: 0.78rem;
  color: var(--neutral-600);
  line-height: 1.5;
}

.toggle {
  position: relative;
  width: 44px;
  height: 24px;
  background: var(--neutral-300);
  border-radius: 12px;
  cursor: pointer;
  transition: background 0.2s;
}

.toggle.active {
  background: var(--primary-600);
}

.toggle-thumb {
```

```

    position: absolute;
    top: 2px;
    left: 2px;
    width: 20px;
    height: 20px;
    background: white;
    border-radius: 50%;
    transition: transform 0.2s;
  }

.toggle.active .toggle-thumb {
  transform: translateX(20px);
}

@media (max-width: 900px) {
  .grid-2, .grid-3, .grid-4 {
    grid-template-columns: 1fr;
  }
  .stats-grid {
    grid-template-columns: repeat(2, 1fr);
  }
  .header h1 {
    font-size: 1.1rem;
  }
  .city-badge {
    display: none;
  }
  .main-content {
    padding: 14px;
  }
}

```

Config vite.config.ts

Vite build configuration — 10 lines

```

import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    host: '0.0.0.0',
    port: 5173,
  },
})

```

JSON tsconfig.json

TypeScript compiler configuration — 21 lines

```

{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",
    "strict": true,
    "noUnusedLocals": false,
    "noUnusedParameters": false,
    "noFallthroughCasesInSwitch": true,
  },
  "include": ["src"],
}

```

HTML index.html

HTML entry point — 17 lines

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/savekart-logo.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
    <title>SAVEKART – Warehouse Location Optimization Platform</title>
    <meta property="og:image" content="https://bolt.new/static/og_default.png">
    <meta name="twitter:card" content="summary_large_image">
    <meta name="twitter:image" content="https://bolt.new/static/og_default.png">
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

JSON package.json

NPM package manifest with all dependencies — 30 lines

```
{
  "name": "savekart",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "preview": "vite preview",
    "test": "vitest run"
  },
  "dependencies": {
    "@supabase/supabase-js": "^2.45.4",
    "leaflet": "^1.9.4",
    "papaparse": "^5.4.1",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-leaflet": "^4.2.1"
  },
  "devDependencies": {
    "@types/leaflet": "^1.9.12",
    "@types/papaparse": "^5.3.14",
    "@types/react": "^18.3.10",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.1",
    "typescript": "^5.6.2",
    "vite": "^5.4.8",
    "vitest": "^2.1.1"
  }
}
```